

POC-01

Secure Batch Ingestion with Azure Data Factory, ADLS Gen2 and Azure SQL

Complete Manual Setup + Python Validation + Terraform Recreation + Screenshot Evidence

Project status	Completed manual POC
Primary services	Azure Data Factory, ADLS Gen2, Azure SQL Database
Security	Microsoft Entra ID, Managed Identity, RBAC
IaC	Terraform mini-lab included
Prepared	30 August 2026

Purpose: a reusable study guide, implementation record, troubleshooting reference and GitHub evidence pack.

How to use this document

This PDF records the manual Azure implementation that was completed successfully, then shows the local Python upload path and the Terraform infrastructure recreation path. The manual implementation uses a simple 5-row CSV with an order_date column. The packaged advanced lab contains an order_ts schema, rejects and watermark logic; those two variants are called out explicitly so they are not accidentally mixed.

Important: If you recreate the lab using Terraform after already creating the same resources manually, use a new Resource Group/unique names, delete the manual lab first, or deliberately import existing resources into Terraform. A normal terraform apply cannot simply take ownership of already-existing Azure resources with matching names.

Contents

1. Project objective and architecture
2. Azure services and naming used
3. Prerequisites and cost guardrails
4. Complete manual Azure configuration
5. Local Python upload and Azure CLI authentication
6. End-to-end execution and verification
7. Troubleshooting and lessons learned
8. Terraform recreation - complete steps
9. Security, GitHub and cleanup checklist
10. Interview and CV-ready summary
11. Appendix A. Screenshot evidence gallery (all supplied images)
12. Appendix B. Terraform source files

1. Project objective and architecture

Business scenario: a retail company receives daily order CSV files. The POC lands raw files in ADLS Gen2, orchestrates ingestion with Azure Data Factory, loads Azure SQL staging, performs an idempotent MERGE into a curated orders table, archives processed files, logs execution status and verifies access through Managed Identity and RBAC.

Architecture

```

Local CSV / Python uploader
    |
    v
ADLS Gen2: landing/orders/YYYY/MM/DD/orders_001.csv
    |
    v
ADF: PL_INGEST_ORDERS_BATCH
  1. Get_Metadata_File
  2. If_File_Exists
  3. Script_Log_Start
  4. Copy_Orders_To_Staging
  5. SP_Merge_Orders
  6. Copy_Archive_File
  7. Delete_Landing_File
  8. Script_Log_Success
    |
    +--> archive/orders/YYYY/MM/DD/...
    |
    v
Azure SQL: dbo.orders_stg -> dbo.orders
    |
    v
ADF Monitor + SQL verification
  
```

Component	Why it exists
Landing zone	Preserves incoming batch data before relational processing.
Staging table	Separates ingestion from final business-key MERGE.
MERGE	Makes business-row loading repeatable without duplicate order_id values.
Archive	Retains processed raw files for audit/replay.
Managed Identity	Lets ADF request Entra tokens without storing a password in pipeline JSON.
RBAC	Controls which identity can read/write/delete ADLS data.

2. Azure services and naming used

Azure object	POC name	Purpose
Resource Group	rg-azde-poc01-dev	Lifecycle boundary and easy cleanup
Storage / ADLS Gen2	stazdepocirshad01	landing, archive and quarantine containers
Data Factory	adf-azde-poc-irshad-01	Batch orchestration
Azure SQL logical server	sql-azde-poc-irshad-01	Hosts the SQL database
Azure SQL database	sqlldb-azde-poc01-dev	Staging, curated orders and ETL log
Containers	landing / archive / quarantine	Raw input, processed files, invalid-data destination
Pipeline	PL_INGEST_ORDERS_BATCH	Main orchestration pipeline
ADLS linked service	LS_ADLS_GEN2_MI	ADF -> ADLS with Managed Identity
SQL linked service	LS_AZURE_SQL_MI	ADF -> Azure SQL with Managed Identity

3. Prerequisites and cost guardrails

- Azure subscription with permission to create resources and role assignments.
- Windows laptop with Python 3.12/Poetry environment for the local uploader.
- Azure CLI for az login and DefaultAzureCredential local authentication.

- Terraform 1.6+ only for the IaC recreation phase.
- Use a small Azure SQL development SKU, run ADF only on demand, and delete the Resource Group after evidence capture.
- Do not commit SQL passwords, terraform.tfvars, .env files, SAS tokens, Storage keys or Terraform state to GitHub.

4. Complete manual Azure configuration

This section is the click-by-click implementation path for rebuilding the POC manually in Azure Portal. It reflects the completed manual setup and the supplied Azure setup guide.

4.1 Create the Resource Group

1. Sign in to Azure Portal and confirm the correct directory and subscription.
2. Search Resource groups -> Create.
3. Create rg-azde-poc01-dev in a region supported by the services you plan to use.
4. Add tags such as project=azure-poc, environment=dev, owner=personal, autoDelete=true.
5. Optionally create a small Cost Management budget and email threshold.

4.2 Create ADLS Gen2 storage

1. Search Storage accounts -> Create and place it in rg-azde-poc01-dev.
2. Use StorageV2, Standard and LRS for the small lab.
3. Enable Hierarchical namespace so the account behaves as ADLS Gen2.
4. Keep public/anonymous blob access disabled.
5. Create private containers: landing, archive and quarantine.
6. Use the folder convention landing/orders/2026/08/28/orders_001.csv.

4.3 Create Azure SQL logical server and database

1. Search SQL databases -> Create.
2. Create database sqldb-azde-poc01-dev.
3. Create logical server sql-azde-poc-irshad-01 and choose an available region.
4. If a region displays a subscription-capacity restriction, choose another supported region; one of the screenshots records this issue.
5. Use a small development tier available in your region and create a bootstrap SQL administrator.
6. Never store the bootstrap SQL password in GitHub.

4.4 Configure SQL networking/firewall

1. Open the Azure SQL logical server -> Networking.
2. Enable the beginner-lab exception Allow Azure services and resources to access this server.
3. Add your current client IP if you need portal/query/client access from the laptop.
4. Save the networking configuration.

4.5 Configure Microsoft Entra admin

1. Open the SQL logical server and configure a Microsoft Entra administrator you control.
2. Connect to sqldb-azde-poc01-dev as that Entra administrator before creating the ADF contained user.

4.6 Create Azure Data Factory

1. Search Data factories -> Create.
2. Create adf-azde-poc-irshad-01 in the POC Resource Group.
3. After deployment, open ADF Studio.
4. ADF uses a system-assigned Managed Identity. This identity is what receives Storage and SQL permissions.

4.7 Give ADF Storage data-plane access

1. Open stazdepocirshad01 -> Access Control (IAM) -> Add role assignment.
2. Assign Storage Blob Data Contributor to the system-assigned Managed Identity of adf-azde-poc-irshad-01.
3. If Python will upload using your local az login, assign the same data role to your own Entra user separately.

4.8 Create SQL tables

Run the following DDL in the completed manual version. This schema uses order_date because it matches the 5-row CSV and the working pipeline.

```
-- ETL file log
IF OBJECT_ID('dbo.etl_file_log', 'U') IS NULL
BEGIN
    CREATE TABLE dbo.etl_file_log (
```

```

        log_id INT IDENTITY(1,1) PRIMARY KEY,
        file_name VARCHAR(255) NOT NULL,
        file_size_bytes BIGINT,
        status VARCHAR(50) NOT NULL,
        rows_inserted INT DEFAULT 0,
        rows_updated INT DEFAULT 0,
        started_at DATETIME2,
        completed_at DATETIME2,
        error_message NVARCHAR(MAX)
    );
END;

-- Staging table
IF OBJECT_ID('dbo.orders_stg', 'U') IS NULL
BEGIN
    CREATE TABLE dbo.orders_stg (
        order_id INT, customer_id INT, order_date VARCHAR(50),
        product_id INT, quantity INT, unit_price DECIMAL(10,2), status VARCHAR(50)
    );
END;

-- Curated target
IF OBJECT_ID('dbo.orders', 'U') IS NULL
BEGIN
    CREATE TABLE dbo.orders (
        order_id INT PRIMARY KEY, customer_id INT, order_date DATE,
        product_id INT, quantity INT, unit_price DECIMAL(10,2),
        total_amount AS (quantity * unit_price) PERSISTED,
        status VARCHAR(50), source_file VARCHAR(255),
        created_at DATETIME2 DEFAULT SYSUTCDATETIME(),
        updated_at DATETIME2 DEFAULT SYSUTCDATETIME()
    );
END;

```

4.9 Create the idempotent MERGE stored procedure

```

CREATE OR ALTER PROCEDURE dbo.usp_merge_orders
    @p_file_name VARCHAR(255)
AS
BEGIN
    SET NOCOUNT ON;

    MERGE dbo.orders AS tgt
    USING (
        SELECT order_id, customer_id, TRY_CONVERT(DATE, order_date) AS order_date,
            product_id, quantity, unit_price, status
        FROM dbo.orders_stg
    ) AS src
    ON tgt.order_id = src.order_id
    WHEN MATCHED THEN
        UPDATE SET tgt.customer_id=src.customer_id, tgt.order_date=src.order_date,
            tgt.product_id=src.product_id, tgt.quantity=src.quantity,
            tgt.unit_price=src.unit_price, tgt.status=src.status,
            tgt.source_file=@p_file_name, tgt.updated_at=SYSUTCDATETIME()
    WHEN NOT MATCHED THEN
        INSERT (order_id, customer_id, order_date, product_id, quantity, unit_price,
            status, source_file, created_at, updated_at)
        VALUES (src.order_id, src.customer_id, src.order_date, src.product_id,
            src.quantity, src.unit_price, src.status, @p_file_name,
            SYSUTCDATETIME(), SYSUTCDATETIME());
END;

```

4.10 Create the ADF database user

Connect to the target database as Microsoft Entra admin and create the contained user for the ADF Managed Identity:

```

CREATE USER [adf-azde-poc-irshad-01] FROM EXTERNAL PROVIDER;
ALTER ROLE db_datareader ADD MEMBER [adf-azde-poc-irshad-01];
ALTER ROLE db_datawriter ADD MEMBER [adf-azde-poc-irshad-01];
ALTER ROLE db_ddladmin ADD MEMBER [adf-azde-poc-irshad-01];

```

Learning note: The completed manual guide uses database roles for simplicity. The advanced package later demonstrates narrower GRANT permissions.

4.11 Create ADF linked services

Name	Type	Authentication	Target
LS_ADLS_GEN2_MI	Azure Data Lake Storage Gen2	System-assigned Managed Identity	stazdepocirshad01.dfs.core.windows.net
LS_AZURE_SQL_MI	Azure SQL Database	System-assigned Managed Identity	sql-azde-poc-irshad-01.database.windows.net / sqldb-azde-poc01-dev

1. ADF Studio -> Manage -> Linked services -> New.
2. Create/test the ADLS linked service using Managed Identity.
3. Create/test the Azure SQL linked service using Managed Identity.
4. Do not place access keys or passwords into exported JSON.

4.12 Create parameterized datasets

Dataset	Type	Parameters	Use
DS_ADLS_OrdersCsv	DelimitedText	p_container, p_folder, p_file	CSV source; first row as header
DS_ADLS_Binary	Binary	p_container, p_folder, p_file	Archive/delete operations
DS_SQL_Table	Azure SQL Database	p_target_table	Schema dbo; table from parameter

```
ADLS CSV path
File system : @dataset().p_container
Directory   : @dataset().p_folder
File        : @dataset().p_file

SQL table
Schema      : dbo
Table       : @dataset().p_target_table
```

4.13 Create pipeline PL_INGEST_ORDERS_BATCH

Parameter	Type	Default
p_container	String	landing
p_folder	String	orders/2026/08/28
p_file	String	orders_001.csv
p_target_table	String	orders_stg

#	Activity	ADF type	Configuration
1	Get_Metadata_File	Get Metadata	Checks Exists, Size, Last modified for parameterized CSV.
2	If_File_Exists	If Condition	Expression: @activity('Get_Metadata_File').output.exists; False -> Fail_File_Not_Found.
3	Script_Log_Start	Script	Insert IN_PROGRESS row into dbo.etl_file_log.
4	Copy_Orders_To_Staging	Copy Data	CSV -> dbo.orders_stg; pre-copy script TRUNCATE TABLE dbo.orders_stg.
5	SP_Merge_Orders	Stored Procedure	Execute dbo.usp_merge_orders with p_file_name.
6	Copy_Archive_File	Binary Copy	landing path -> archive path.
7	Delete_Landing_File	Delete	Delete successfully processed source from landing.
8	Script_Log_Success	Script	Update ETL log to SUCCESS with row count/completion time.

If condition expression

```
@activity('Get_Metadata_File').output.exists
```

Fail message expression

```
@concat('File does not exist: ', pipeline().parameters.p_container, '/',  
        pipeline().parameters.p_folder, '/', pipeline().parameters.p_file)
```

Script_Log_Start

```
INSERT INTO dbo.etl_file_log (file_name, file_size_bytes, status, started_at)  
VALUES ('@{pipeline().parameters.p_file}', @{activity('Get_Metadata_File').output.size},  
        'IN_PROGRESS', SYSUTCDATETIME());
```

Copy activity pre-copy script

```
TRUNCATE TABLE dbo.orders_stg;
```

Script_Log_Success

```
UPDATE dbo.etl_file_log  
SET status='SUCCESS',  
    rows_inserted=(SELECT COUNT(1) FROM dbo.orders  
                   WHERE source_file='@{pipeline().parameters.p_file}'),  
    rows_updated=0, completed_at=SYSUTCDATETIME()  
WHERE file_name='@{pipeline().parameters.p_file}' AND status='IN_PROGRESS';
```

4.14 Validate, publish and run

1. Click Validate all and resolve errors until validation is clean.
2. Click Publish all.
3. Upload the source CSV to the exact landing folder.
4. Use Debug or Add trigger -> Trigger now.
5. Open Monitor -> Pipeline runs and inspect every activity output.

5. Local Python upload and Azure CLI authentication

The completed POC can use Python only for landing the CSV. The existing working ADF pipeline remains unchanged. The Python Azure SDK packages do not install the az command; Azure CLI is a separate Windows installation.

5.1 Azure CLI login

```
winget install --exact --id Microsoft.AzureCLI

# Close all terminals, open a new CMD/PowerShell, then:
az version
az login
az account show -o table
az group show --name rg-azde-poc01-dev -o table
```

Note: If 'az' is not recognized after installation, close all terminals and open a new one so Windows refreshes PATH. Use 'where az' to verify the executable is discoverable.

5.2 Python environment

```
poetry run python --version
poetry run python -c "import azure.identity; import azure.storage.filedatalake; print('Azure Python SDK OK')"
```

Required packages used in the existing common Poetry environment:

```
azure-identity = "^1.25.3"
azure-storage-file-datalake = "^12.25.0"
```

5.3 Local 5-row CSV for the manual pipeline

```
order_id,customer_id,order_date,product_id,quantity,unit_price,status
1001,501,2026-08-28,10,2,25.50,Completed
1002,502,2026-08-28,12,1,100.00,Pending
1003,503,2026-08-28,15,4,15.75,Completed
1004,504,2026-08-28,10,1,25.50,Shipped
1005,505,2026-08-28,20,3,45.00,Completed
```

Schema compatibility: The packaged generate_orders.py uses the advanced order_ts schema. For the completed manual order_date pipeline, use this 5-row CSV unless you intentionally migrate to the advanced schema.

5.4 Upload to ADLS using DefaultAzureCredential

```
poetry run python python\upload_to_adls.py ^
--storage-account stazdepocirshad01 ^
--container landing ^
--remote-path orders/2026/08/28/orders_001.csv ^
--local-file data/generated/orders_001.csv ^
--overwrite
```

Authentication flow: az login -> Microsoft Entra token -> DefaultAzureCredential -> DataLakeServiceClient -> ADLS Gen2. No Storage key or SAS token is required in the script.

6. End-to-end execution and verification

6.1 First successful run

1. Confirm orders_001.csv exists under landing/orders/2026/08/28/.
2. Run PL_INGEST_ORDERS_BATCH.
3. Confirm all expected ADF activities succeed.
4. Query dbo.orders and dbo.etl_file_log.
5. Verify source file moved to archive and no longer exists in landing.

```
SELECT * FROM dbo.orders ORDER BY order_id;
SELECT COUNT(*) AS total_curated_orders FROM dbo.orders;
SELECT * FROM dbo.etl_file_log ORDER BY log_id DESC;
```

Pass condition: dbo.orders contains exactly 5 business rows and the latest ETL log row is SUCCESS.

6.2 Idempotency test

1. Re-upload the exact same orders_001.csv to the same landing path.
2. Run the ADF pipeline again.
3. Query the curated count.

```
SELECT COUNT(*) AS total_curated_orders FROM dbo.orders;
```

Pass condition: Count remains 5, not 10. MERGE updates/matches on order_id instead of blindly appending duplicates.

6.3 Archive and missing-file test

- After success, archive/orders/2026/08/28/orders_001.csv should exist.
- The landing copy should be deleted.
- Run ADF again without re-uploading. If_File_Exists should go to Fail_File_Not_Found, proving a controlled missing-input failure.

6.4 RBAC controlled-failure test

1. Re-upload the test file.
2. Temporarily remove Storage Blob Data Contributor from the ADF Managed Identity only.
3. Run ADF and capture the authorization/403 failure.
4. Restore the role.
5. Retry and verify the pipeline succeeds.

Pass condition: Removing ADF data-plane permission causes a controlled storage access failure; restoring RBAC allows the same pipeline to succeed.

6.5 Monitor evidence

- ADF Studio -> Monitor -> Pipeline runs.
- Open Copy_Orders_To_Staging and confirm Rows read = 5 and Rows written = 5 for the sample.
- Review activity duration, status and error details for failed tests.
- Capture SQL row counts and ETL log as independent verification.

Important verification note: Manual implementation staging behavior: the manual stored procedure shown in this guide does not delete staging rows after MERGE. The Copy Activity truncates staging before the next load. Therefore, immediately after a successful manual run, dbo.orders_stg can still contain 5 rows. The advanced package version uses a different procedure that deletes the file/run staging rows after MERGE.

7. Troubleshooting and lessons learned

Symptom	Check / fix
az is not recognized	azure-identity is a Python SDK, not Azure CLI. Install Microsoft.AzureCLI separately, reopen terminal, verify az version.
Azure SQL region/capacity restriction	Choose a supported region/available capacity for the subscription. A supplied screenshot records this exact issue.
Script_Log_Start times out	Check SQL server firewall and confirm Allow Azure services/resources is enabled for this beginner public-endpoint lab.
SQL Managed Identity login fails	Set a Microsoft Entra admin, connect to the target database, CREATE USER for the exact ADF name, then grant required database permissions.
ADF Get Metadata says file absent	Verify container/folder/file parameters exactly match landing/orders/2026/08/28/orders_001.csv.
Python upload returns 403	Assign Storage Blob Data Contributor to the signed-in developer user, not only to ADF.
ADF storage access returns 403	Assign/restore Storage Blob Data Contributor to the ADF system-assigned Managed Identity.
Duplicate rerun concerns	MERGE uses order_id as business key; count should remain 5 for the completed manual sample.
Archive copy succeeds but landing remains	Check Delete activity dependency, original landing dataset parameters and ADF delete permission.

8. Terraform recreation - complete steps

The Terraform folder in the project is an infrastructure mini-lab. It creates the Azure foundation but deliberately leaves tenant-specific Microsoft Entra SQL administration, SQL schema objects and ADF authoring as post-deployment learning steps.

Before terraform apply: Because the manual POC is already deployed, the simplest Terraform practice is either (A) delete the manual POC and recreate it, or (B) use a second Resource Group and unique globally scoped names. Importing the existing manually created resources into Terraform is an advanced alternative.

8.1 What the supplied Terraform creates

Terraform resource	Purpose
azurerm_resource_group	POC Resource Group
azurerm_storage_account	StorageV2, HNS enabled, TLS 1.2
azurerm_storage_container	landing, archive, quarantine
azurerm_data_factory	ADF with system-assigned identity
azurerm_role_assignment	ADF -> Storage Blob Data Contributor
azurerm_role_assignment	Optional developer -> Storage Blob Data Contributor
azurerm_key_vault	RBAC-enabled Key Vault
azurerm_role_assignment	ADF -> Key Vault Secrets User
azurerm_mssql_server	Azure SQL logical server
azurerm_mssql_database	Basic 2 GB lab database
azurerm_mssql_firewall_rule	Allow Azure services + optional developer IP
azurerm_log_analytics_workspace	Optional, controlled by create_log_analytics

Scope: The supplied Terraform does not create ADF linked services, datasets, activities/pipeline JSON or SQL tables/stored procedures. Complete those after infrastructure deployment using the manual sections or the packaged ADF/SQL artifacts.

8.2 Terraform prerequisites

1. Install Terraform 1.6 or newer and verify: terraform -version.
2. Install Azure CLI and authenticate: az login.
3. Confirm the intended subscription: az account show -o table.
4. Change directory to infra/terraform.

```
cd infra\terraform
az login
az account show -o table
terraform -version
```

8.3 Create terraform.tfvars

Copy-Item terraform.tfvars.example terraform.tfvars

Edit terraform.tfvars locally. Use unique names if the manual POC still exists:

```
subscription_id = "<YOUR_SUBSCRIPTION_ID>"
location        = "centralindia"
resource_group_name = "rg-azde-poc01-tf-dev"

storage_account_name = "stazdepoc01UNIQUE"
data_factory_name    = "adf-azde-poc01-UNIQUE"
sql_server_name      = "sql-azde-poc01-UNIQUE"
sql_database_name    = "sqladb-azde-poc01-dev"
key_vault_name       = "kv-azde-poc01-UNIQUE"

sql_admin_login      = "pocsqladmin"
sql_admin_password   = "CHANGE_ME_LOCALLY"

client_ip = null
uploader_object_id = null
```

```
create_log_analytics = false
```

Git safety: terraform.tfvars and *.tfstate are excluded by .gitignore and should never be committed because they can contain sensitive values.

8.4 Initialize, format, validate and plan

```
terraform init
terraform fmt -recursive
terraform validate
terraform plan
```

- Read the plan carefully before apply.
- Confirm the target subscription, Resource Group and resource names.
- If Azure reports a resource already exists, stop and resolve the manual-vs-Terraform ownership issue instead of forcing changes.

8.5 Apply

```
terraform apply
```

Review the displayed plan and type the Terraform confirmation only when the resource list is correct.

8.6 Inspect outputs and Azure resources

```
terraform output
az resource list --resource-group rg-azde-poc01-tf-dev -o table
```

Expected outputs include Resource Group name, storage account/DFS URL, Data Factory name and Managed Identity object ID, SQL server FQDN, SQL database name and Key Vault URI.

8.7 Required post-deployment SQL steps

1. Open the Terraform-created SQL logical server.
2. Set a Microsoft Entra administrator you control.
3. Connect to the target database as that Entra admin.
4. For an exact reproduction of the completed manual 5-row version, run the manual table/procedure/user SQL shown in Sections 4.8-4.10.
5. Alternatively, if you intentionally switch to the advanced package schema, use sql/001_create_tables.sql, sql/002_merge_orders.sql and sql/003_create_adf_user.sql together with the advanced order_ts ADF mapping. Do not mix schemas.

8.8 Required post-deployment ADF steps

1. Open the Terraform-created Data Factory.
2. Create LS_ADLS_GEN2_MI and LS_AZURE_SQL_MI.
3. Create DS_ADLS_OrdersCsv, DS_ADLS_Binary and DS_SQL_Table.
4. Create PL_INGEST_ORDERS_BATCH and its parameters/activities using Section 4.
5. Validate all, Publish all and trigger a test run.

8.9 Optional developer Python upload role

The Terraform variable uploader_object_id can grant Storage Blob Data Contributor to your developer user so upload_to_adls.py works with az login. If left null, assign the role manually later.

8.10 Optional Log Analytics

```
create_log_analytics = true
```

With this variable enabled, the supplied Terraform creates a small Log Analytics workspace. Diagnostic settings themselves are a follow-up exercise; keep it disabled if you want minimum lab cost.

8.11 Destroy / cleanup Terraform resources

```
terraform destroy
```

9. Security, GitHub and cleanup checklist

- Use Managed Identity for ADF -> ADLS and ADF -> Azure SQL authentication.
- Use Storage Blob Data Contributor only where data read/write/delete is required.
- Do not commit .env, terraform.tfvars, *.tfstate, Storage keys, SAS tokens, SQL passwords or client secrets.
- Before publishing screenshots to GitHub, consider redacting personal email, subscription identifiers and tenant-specific details visible in the Azure portal.
- For a hardened enterprise version, replace public SQL/Storage access with ADF Managed Virtual Network, managed private endpoints and Private Link.
- After capturing evidence, delete the isolated POC Resource Group or run terraform destroy for the Terraform lab.

```
git status
git diff --cached
# Verify no secrets before commit/push.
```

10. Interview and CV-ready summary

10.1 What this POC demonstrates

- Azure Data Factory orchestration and parameterization
- ADLS Gen2 landing/archive design
- Azure SQL staging and curated model
- Idempotent SQL MERGE by business key
- Microsoft Entra ID, Managed Identity and RBAC
- File-existence validation and controlled failure handling
- ADF Monitor and SQL execution evidence
- Python SDK upload using DefaultAzureCredential
- Terraform-based Azure infrastructure recreation
- Cost-conscious cleanup and Git-safe configuration

10.2 Rapid interview questions

Question	Short answer
Why land in ADLS before SQL?	It decouples file arrival from relational processing, preserves raw evidence and supports replay/audit.
What makes this pipeline idempotent?	The curated MERGE matches order_id, so a rerun updates/matches existing business rows rather than blindly inserting duplicates.
Why use staging?	It isolates ingestion/type loading from curated-table business logic and makes validation/merge easier to control.
Managed Identity vs password?	Managed Identity lets ADF obtain Entra tokens without storing reusable credentials in pipeline configuration.
Why archive processed files?	To retain the raw source for audit, support replay and keep the landing zone focused on active inputs.
What did the RBAC negative test prove?	Removing the ADF storage data role caused a controlled authorization failure, demonstrating that access is identity/role based.
What does Terraform add?	Repeatable, reviewable infrastructure provisioning; the POC still requires post-deploy SQL/ADF authoring unless those artifacts are also codified.

10.3 CV wording - use after completion

- Built a parameterized Azure Data Factory batch ingestion pipeline to process order CSV files from ADLS Gen2 into Azure SQL staging and curated tables.
- Implemented idempotent SQL MERGE logic, raw-file archival, file-existence checks, execution logging and retry-safe validation scenarios.
- Secured Azure data access with Microsoft Entra ID, system-assigned Managed Identity and RBAC, including controlled permission-failure testing.
- Automated local ADLS uploads with Python Azure SDK and DefaultAzureCredential and recreated core Azure infrastructure with Terraform.

Appendix A. Screenshot evidence gallery

All 42 screenshots supplied in screenshot.zip are included below. The gallery is evidence of resource creation, ADF authoring, troubleshooting, Azure CLI setup, RBAC configuration and final successful execution. Some screenshots are timestamp-named; captions describe the visible stage at a high level without inventing details not shown.

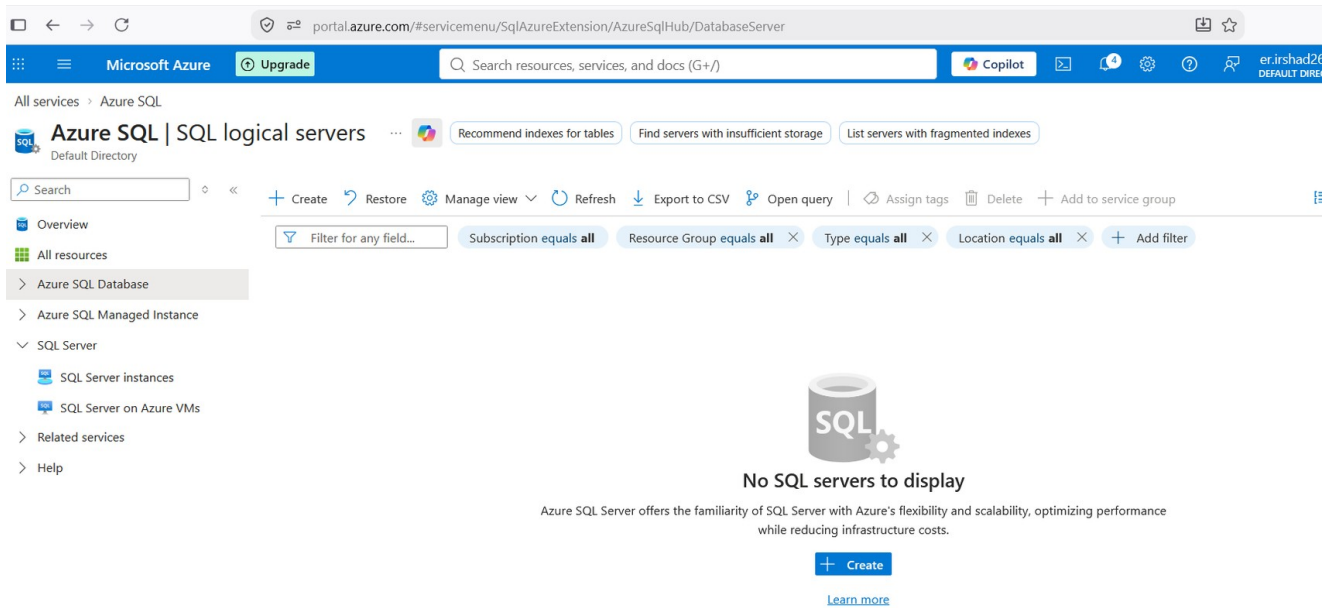


Figure A-1. Azure SQL logical servers service page during SQL setup. [Source file: azure_sql_server.png]

☰

☰

Microsoft Azure

⬆️ Upgrade

🔍 Search resources, services, and docs (G+)

All services > Azure SQL | SQL logical servers

Create SQL Database Server ...

Microsoft

⚠️ Changing Basic options may reset selections you have made. Review all options prior to creating the resource.

manage all your resources.

Subscription * ⓘ

Azure subscription 1

Resource group * ⓘ

rg-azde-poc-dev

[Create new](#)

Server details

Enter required settings for this server, including providing a name and location.

Server name *

sql-azde-poc-irshad-01

✓

.database.windows.net

Location *

(US) East US

✗ Your subscription does not have access to create a server in the selected region. For the latest information about region availability for your subscription, go to aka.ms/sqlcapacity. Please try another region or create a support ticket to request access.

Authentication

ℹ️ Azure Active Directory (Azure AD) is now Microsoft Entra ID. [Learn more](#)

Review + create

Next : Networking >

Figure A-2. Azure SQL server creation - region/capacity restriction encountered during manual setup. [Source file: error_in_creating_database_region.png]

The screenshot shows the Azure portal interface for a deployment named `Microsoft.SqlServer.createServer_d977d2fd25424780a850884fc3be25f`. The deployment is complete, as indicated by a green checkmark and the message "Your deployment is complete". The deployment details show it was created on 8/28/2026 at 4:29:56 PM under the subscription "Azure subscription 1" and resource group "rg-azde-poc-dev". The left sidebar shows the deployment overview, inputs, outputs, and template. The top navigation bar includes the Microsoft Azure logo, an upgrade button, a search bar, and a Copilot button.

Figure A-3. Azure SQL logical server deployment completed. [Source file: `sql_server_created.png`]

The screenshot shows the "Create SQL Database" configuration page in the Azure portal. The page is for the resource `Microsoft.SqlServer.createServer_d977d2fd25424780a850884fc3be25f` in the resource group `sql-azde-poc-irshad-01`. The configuration includes a dropdown for the subscription (set to "Azure subscription 1") and a dropdown for the resource group (set to "rg-azde-poc-dev"). The "Database details" section includes a text input for the database name (placeholder: "Enter database name"), a dropdown for the server (set to "sql-azde-poc-irshad-01 (Central India)"), and radio buttons for "Want to use SQL elastic pool?" (set to "No") and "Workload environment" (set to "Production"). A blue information box at the bottom states: "Default settings provided for Production workloads. Configurations can be modified as needed." The right sidebar shows the estimated compute cost of 586.70 USD per month and storage cost of 0.28 USD per month. The bottom navigation bar includes a "Review + create" button and a "Next: Networking >" button.

Figure A-4. Create SQL Database configuration page. [Source file: `sql_database.png`]

The screenshot displays the Azure portal interface. At the top, the breadcrumb navigation shows 'portal.azure.com/#view/Microsoft_Azure_Resources/DeploymentDetails.MenuView/~/overview/id/%2Fsubscriptions%2F9902e04b-fd65-40f4-86a2-...'. The main header includes the 'Microsoft Azure' logo, an 'Upgrade' button, and a search bar. Below the header, the page title is 'Microsoft.SqlDatabase.newDatabaseExistingServer_e5c6becaece441b5 | Overview'. The left sidebar contains a search bar and a list of tabs: 'Overview' (selected), 'Inputs', 'Outputs', and 'Template'. The main content area features a green checkmark icon and the text 'Your deployment is complete'. Below this, deployment details are listed: 'Deployment name : Microsoft.SqlDatabase.newDatabaseExistingServer_e...', 'Subscription : Azure subscription 1', and 'Resource group : rg-azde-poc-dev'. The 'Start time' is '8/28/2026, 4:36:00 PM' and the 'Correlation ID' is 'd29b3988-a668-4ad5-8608-6fc155105a15'. At the bottom, there are expandable sections for 'Deployment details' and 'Next steps', and a 'Go to resource' button.

Microsoft Azure Upgrade Search resources, services, and docs (G+/)

All services

Microsoft.SqlDatabase.newDatabaseExistingServer_e5c6becaece441b5 | Overview

Deployment

Search x <<

Delete Cancel Redeploy Download Refresh

Overview

Inputs

Outputs

Template

✓ Your deployment is complete

Deployment name : Microsoft.SqlDatabase.newDatabaseExistingServer_e... Start time : 8/28/2026, 4:36:00 PM

Subscription : Azure subscription 1 Correlation ID : d29b3988-a668-4ad5-8608-6fc155105a15

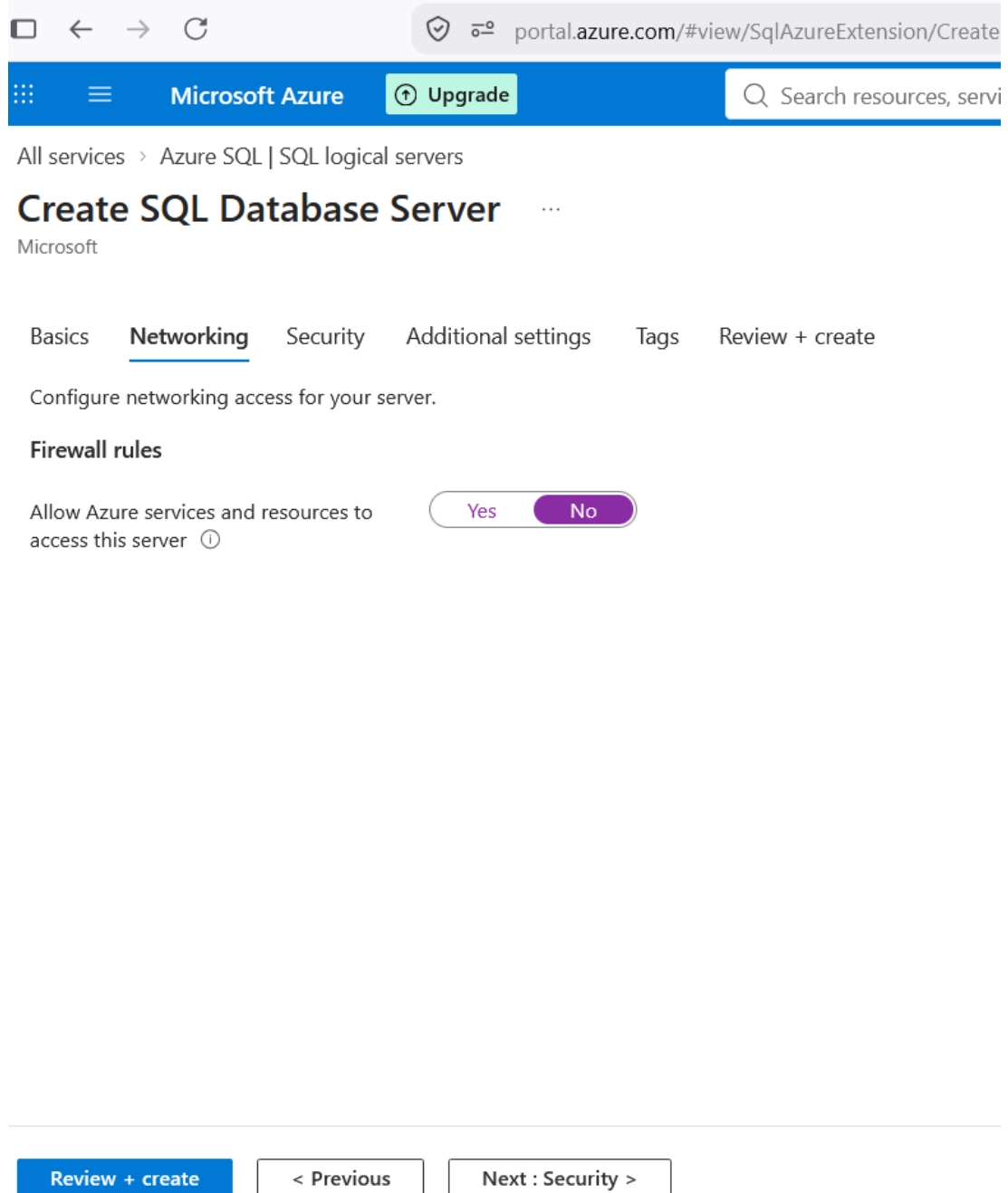
Resource group : rg-azde-poc-dev

> Deployment details

< Next steps

Go to resource

Figure A-5. Azure SQL Database deployment completed. [Source file: database_created.png]



The screenshot shows the 'Create SQL Database Server' page in the Azure portal, specifically the 'Networking' tab. The page title is 'Create SQL Database Server' with a Microsoft logo. The breadcrumb trail is 'All services > Azure SQL | SQL logical servers'. The 'Networking' tab is selected, with other tabs being 'Basics', 'Security', 'Additional settings', 'Tags', and 'Review + create'. Below the tabs, it says 'Configure networking access for your server.' The 'Firewall rules' section has a toggle for 'Allow Azure services and resources to access this server' with 'Yes' and 'No' options. At the bottom, there are three buttons: 'Review + create' (blue), '< Previous' (white), and 'Next : Security >' (white).

POC-01 | Secure Batch Ingestion with Azure Data Factory, ADLS Gen2 and Azure SQL

portal.azure.com/#view/SqlAzureExtension/Create

Microsoft Azure Upgrade Search resources, servi

All services > Azure SQL | SQL logical servers

Create SQL Database Server

Microsoft

Basics **Networking** Security Additional settings Tags Review + create

Configure networking access for your server.

Firewall rules

Allow Azure services and resources to access this server ⓘ

Yes No

Review + create < Previous Next : Security >

Figure A-6. Azure SQL server Networking page - beginner-lab Azure services firewall option. [Source file: sql_server_allow_firewall.png]

POC-01 | Secure Batch Ingestion with Azure Data Factory, ADLS Gen2 and Azure SQL

portal.azure.com/#view/Microsoft_Azure_DataFactory/CreateDataFactory_dx/_provisioningContext~/["initialValues"%3A["subscriptionIds"%3A[]%2C"r...

Microsoft Azure Upgrade Search resources, services, and docs (G+/) Copilot

All services > Data factories

Create Data Factory

Basics Git configuration Networking Advanced Tags Review + create

Project details

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription * ⓘ Azure subscription 1

Resource group * ⓘ Create new

Instance details

Name * ⓘ

Region * ⓘ (US) East US

Version * ⓘ V2

i We recommend you **upgrade to Fabric Data Factory** for a modern, unified data integration experience. [Try Fabric instead!](#)

Previous Next Review + create

Figure A-7. Create Data Factory - Basics configuration. [Source file: data_factory_1.png]

portal.azure.com/#view/Microsoft_Azure_Resources/DeploymentDetails.MenuView/~/-/overview/id/%2Fsubscriptions%2F9902e04b-fd65-40f4-86a2-1b...

Microsoft Azure Upgrade Search resources, services, and docs (G+/) Copilot

All services

CreateDataFactory-20260828151044 | Overview

Deployment

Search x << Delete Cancel Redeploy Download Refresh

Overview

- Inputs
- Outputs
- Template

✓ Your deployment is complete

Deployment name : CreateDataFactory-20260828151044 Start time : 8/28/2026, 3:16:35 PM

Subscription : Azure subscription 1 Correlation ID : d3e345d7-e13a-47fc-9fd3-c0b92c20682c

Resource group : rg-azde-poc-dev

> Deployment details

✓ Next steps

[Go to resource](#)

Cor Get pre Set

Mic Sec Go

Fre Sta

Wo

Figure A-8. Azure Data Factory deployment completed. [Source file: data_factory_2.png]

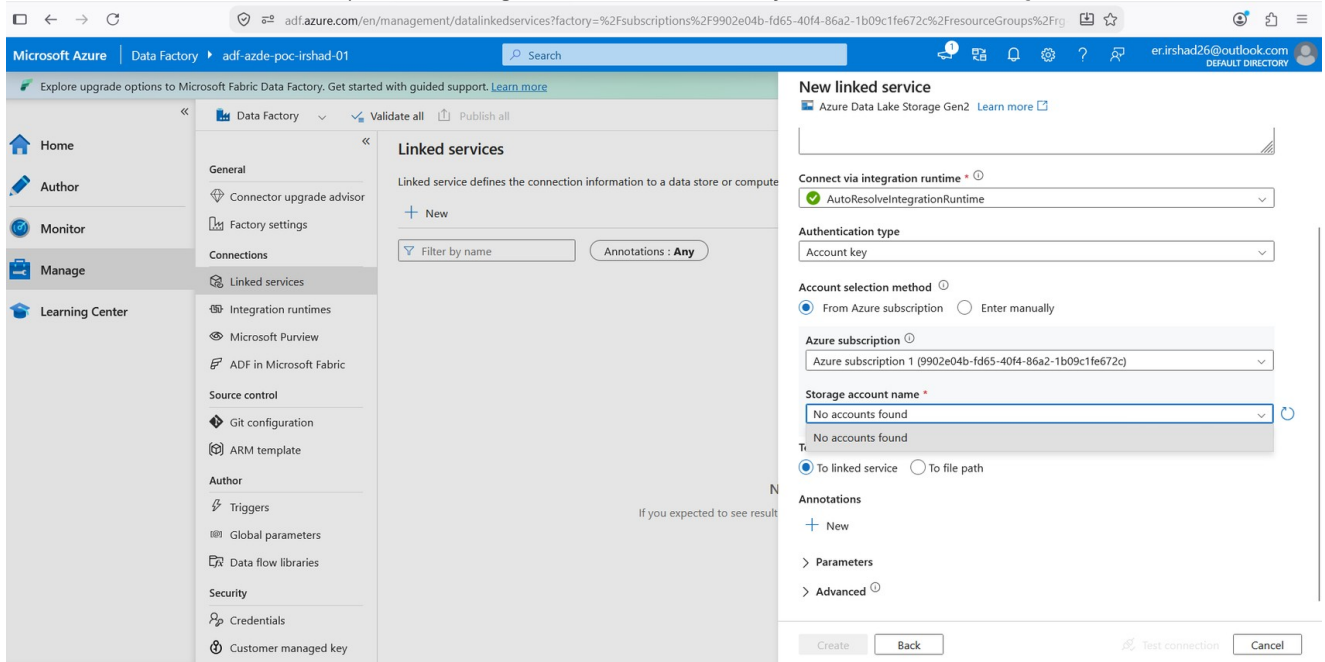


Figure A-9. ADF new linked service configuration for ADLS Gen2 using Managed Identity. [Source file: data_factory_3.png]

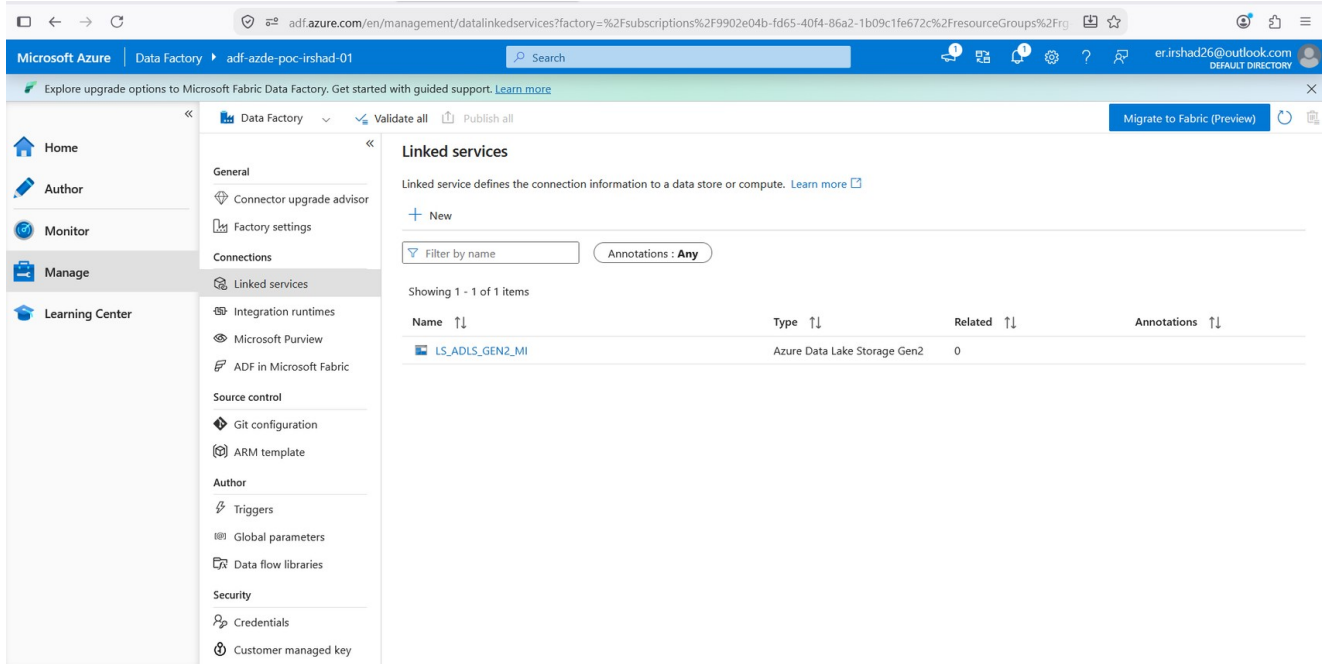
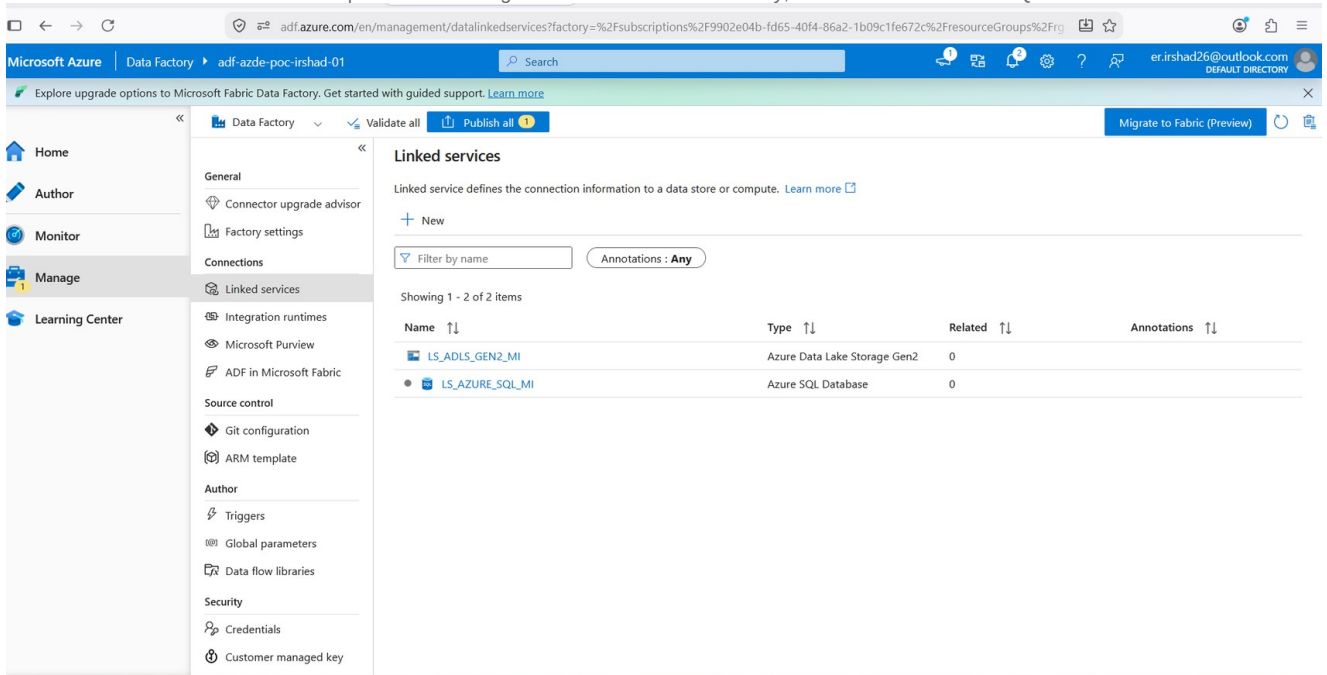


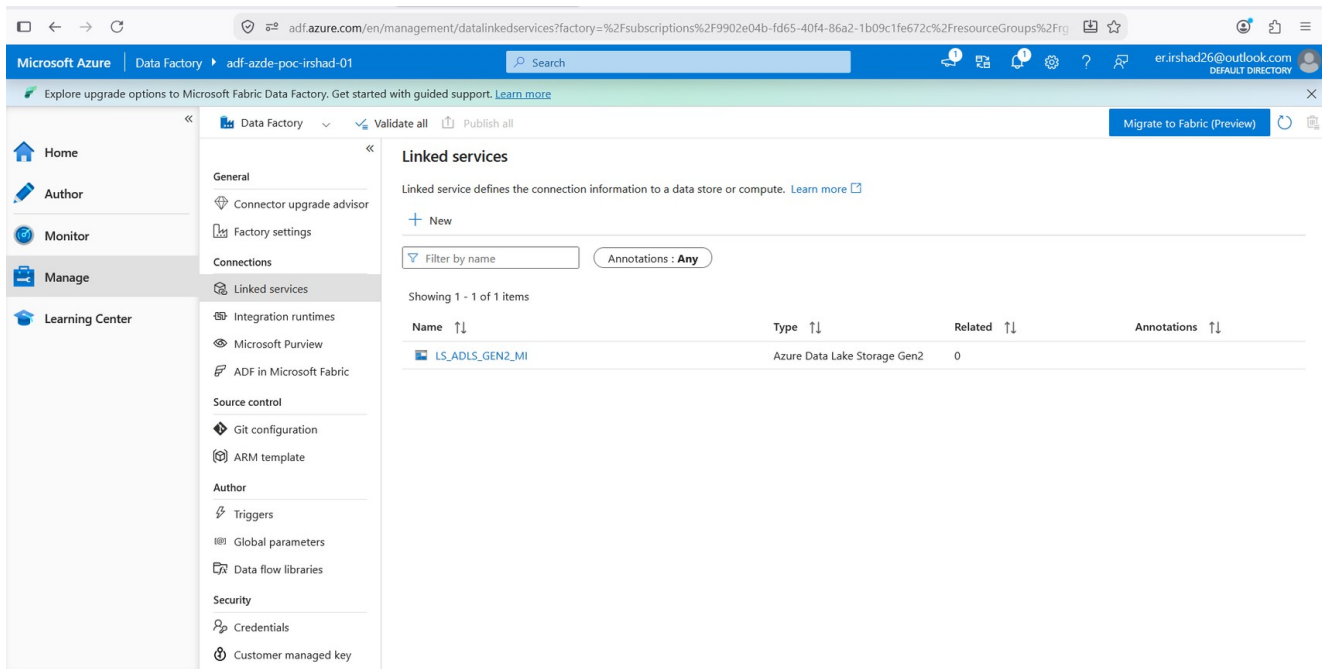
Figure A-10. ADF Manage -> Linked services showing the ADLS linked service. [Source file: data_factory_4.png]



The screenshot shows the Azure Data Factory portal interface. The left sidebar contains navigation options: Home, Author, Monitor, Manage (selected), and Learning Center. The 'Manage' section is expanded, showing various settings like General, Connections, Source control, and Security. The 'Connections' section is selected, displaying a list of linked services. The main pane shows the 'Linked services' page with a table listing two services:

Name	Type	Related	Annotations
LS_ADLS_GEN2_MI	Azure Data Lake Storage Gen2	0	
LS_AZURE_SQL_MI	Azure SQL Database	0	

Figure A-11. ADF linked-services list after adding Azure SQL connectivity. [Source file: data_factory_linked_database.png]



The screenshot shows the same Azure Data Factory portal interface, but the 'Linked services' list now only contains one service:

Name	Type	Related	Annotations
LS_ADLS_GEN2_MI	Azure Data Lake Storage Gen2	0	

Figure A-12. ADF linked-services view showing Azure SQL linked service. [Source file: linked_service_sql_server.png]

Microsoft Azure | Upgrade | Search resources, services, and docs (G+)

er.irshad26@outlook.com

stazdepocirshad01 | Access Control (IAM)

Check access | **Role assignments** | Roles | Deny assignments

Number of role assignments for this subscription: 2 / 5000

Search by name or email | Type: All | Role: All | Scope: All scopes | Group by: Role

All (2) | Job function roles (1) | Privileged administrator roles (1)

Name	Type	Role	Scope	Condition
Owner (1)				
irshad alam 0cabca04-45aa-440e-9ce9-4b0...	User	Owner	Subscription (Inherited)	None
Storage Blob Data Contributor (1)				
adf-azde-poc-irshad-01 2c4991cc-10bc-4412-8d7c-7d...	Managed identity	Storage Blob Data Contributor	This resource	Add

Showing 1 - 2 of 2 results.

Figure A-13. Storage Account IAM role assignments showing user/managed-identity access. [Source file: assing_permission.png]

Microsoft Azure | Upgrade | Search resources, services, and docs (G+)

er.irshad26@outlook.com

stazdepocirshad01 | Access Control (IAM)

Check access | **Role assignments** | Roles | Deny assignments

Number of role assignments for this subscription: 2 / 5000

Search by name or email | Type: All | Role: All | Scope: All scopes | Group by: Role

All (2) | Job function roles (1) | Privileged administrator roles (1)

Name	Type	Role	Scope	Condition
Owner (1)				
irshad alam 0cabca04-45aa-440e-9ce9-4b0...	User	Owner	Subscription (Inherited)	None
Storage Blob Data Contributor (1)				
adf-azde-poc-irshad-01 2c4991cc-10bc-4412-8d7c-7d...	Managed identity	Storage Blob Data Contributor	This resource	Add

Showing 1 - 2 of 2 results.

Figure A-14. Storage Account IAM evidence for Storage Blob Data Contributor role assignments. [Source file: storage.png]

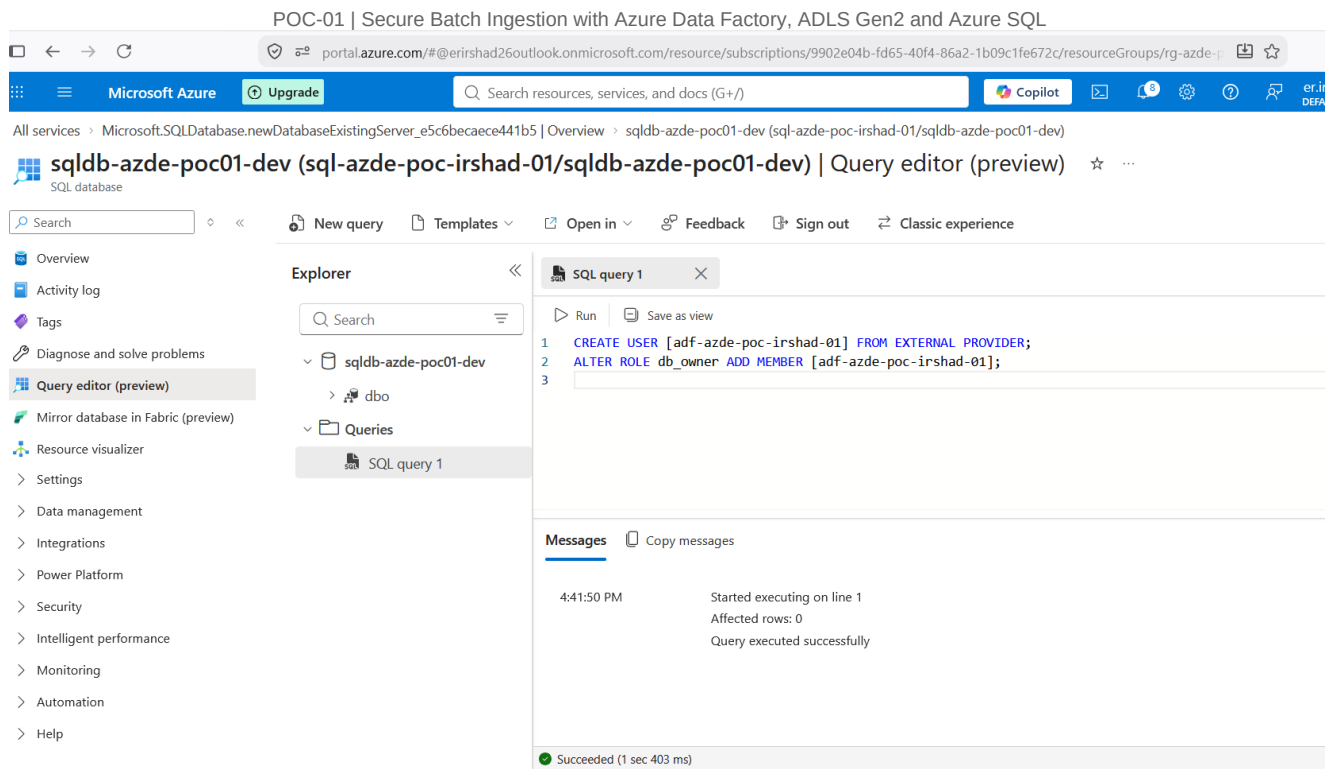


Figure A-15. Azure SQL Query Editor - CREATE USER / role permission setup for ADF Managed Identity. [Source file: `allow_ADF_permission_script.png`]

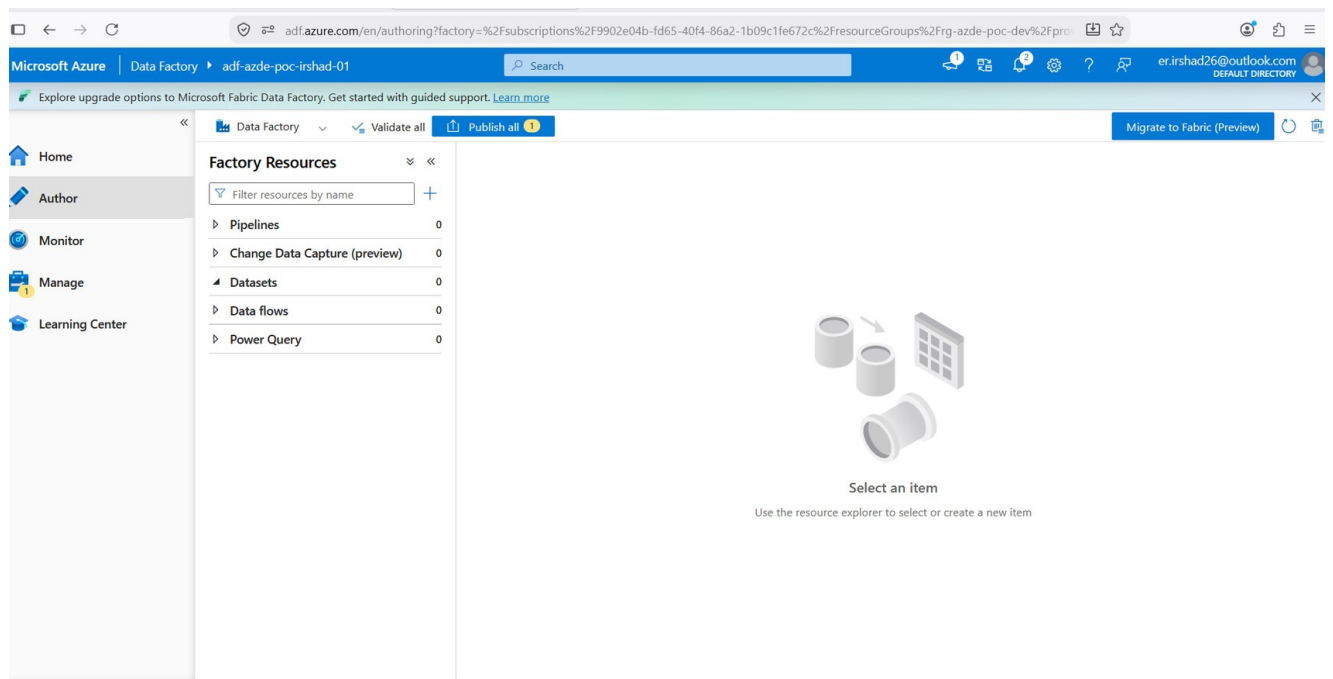


Figure A-16. ADF Studio Author view at the start of dataset/pipeline authoring. [Source file: `Screenshot 2026-08-28 172112.png`]

The screenshot shows the 'Set properties' dialog for creating a new dataset in Azure Data Factory. The dialog is titled 'Set properties' and contains the following fields and options:

- Name:** DS_ADLS_OrdersCsv
- Linked service:** LS_ADLS_GEN2_MI
- File path:** File system / Directory / File name
- First row as header:** ☒
- Import schema:** ☐ From connection/store ☐ From sample file ☒ None

The 'Factory Resources' pane on the left shows the following resources:

- Pipelines: 0
- Change Data Capture (preview): 0
- Datasets: 0**
- Data flows: 0
- Power Query: 0

The 'Set properties' dialog has 'OK', 'Back', and 'Cancel' buttons at the bottom.

Figure A-17. ADF dataset creation properties during manual authoring. [Source file: Screenshot 2026-08-28 172438.png]

er.irshad26@outlook.com
DEFAULT DIRECTORY

Set properties

Name

DS_ADLS_OrdersCsv

Linked service *

LS_ADLS_GEN2_MI

File path

@dataset().p_container / @dataset().p_folder / @dataset().p_file

First row as header ☒

Import schema

☒ From connection/store ☐ From sample file ☐ None

From root
From specified path

Figure A-18. DS_ADLS_OrdersCsv properties and parameterized file path. [Source file: Screenshot 2026-08-28 172820.png]

port. [Learn more](#)

Publish all 2 Migrate to Fabric (Preview)

DS_ADLS_OrdersCsv



DelimitedText
DS_ADLS_OrdersCsv

Connection Schema Parameters

Linked service * LS_ADLS_GEN2_MI

[Test connection](#) [Edit](#) [+ New](#) [Learn more](#)

File path @dataset().p_container / @dataset().p_folder

Compression type No compression

Column delimiter Comma (,)

Row delimiter Default (\r\n, or \r\n)

Encoding Default(UTF-8)

Quote character Double quote (")

Properties

General Related

Name * DS_ADLS_OrdersCsv

Description

Annotations [+ New](#)

Cancel

Figure A-19. DelimitedText ADLS orders dataset connection settings. [Source file: Screenshot 2026-08-28 173215.png]

Data Factory Validate all Publish all 3 Migrate to Fabric (Preview)

Factory Resources

- Pipelines 0
- Change Data Capture (preview) 0
- Datasets 2**
 - DS_ADLS_Binary**
 - DS_ADLS_OrdersCsv
- Data flows 0
- Power Query 0



Binary
DS_ADLS_Binary

Connection Parameters

Linked service * LS_ADLS_GEN2_MI

[Test connection](#) [Edit](#) [+ New](#) [Learn more](#)

Connection successful

File path @dataset().p_container / @dataset().p_folder

Compression type No compression

Properties

General Related

Name * DS_ADLS_Binary

Description

Annotations [+ New](#)

Cancel

Figure A-20. Binary ADLS dataset connection settings for archive/delete operations. [Source file: Screenshot 2026-08-28 173719.png]

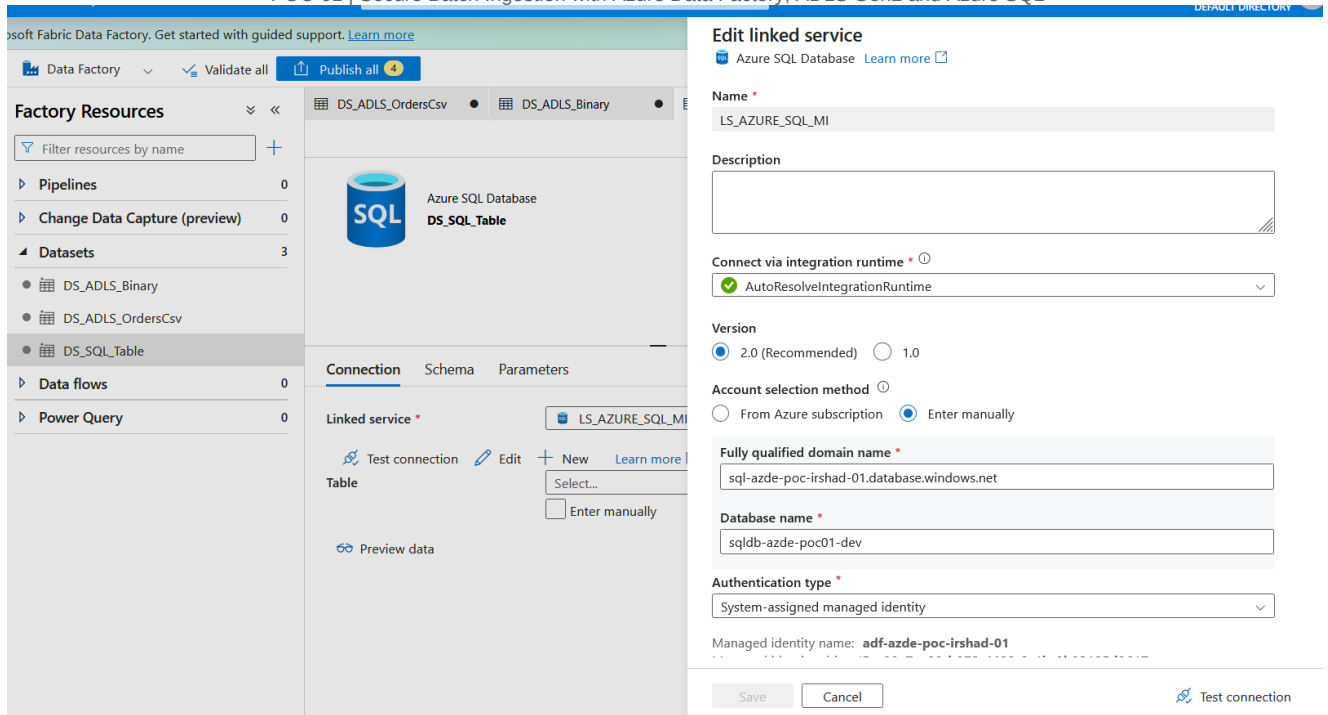


Figure A-21. ADF Azure SQL linked-service configuration using Managed Identity. [Source file: Screenshot 2026-08-28 174147.png]

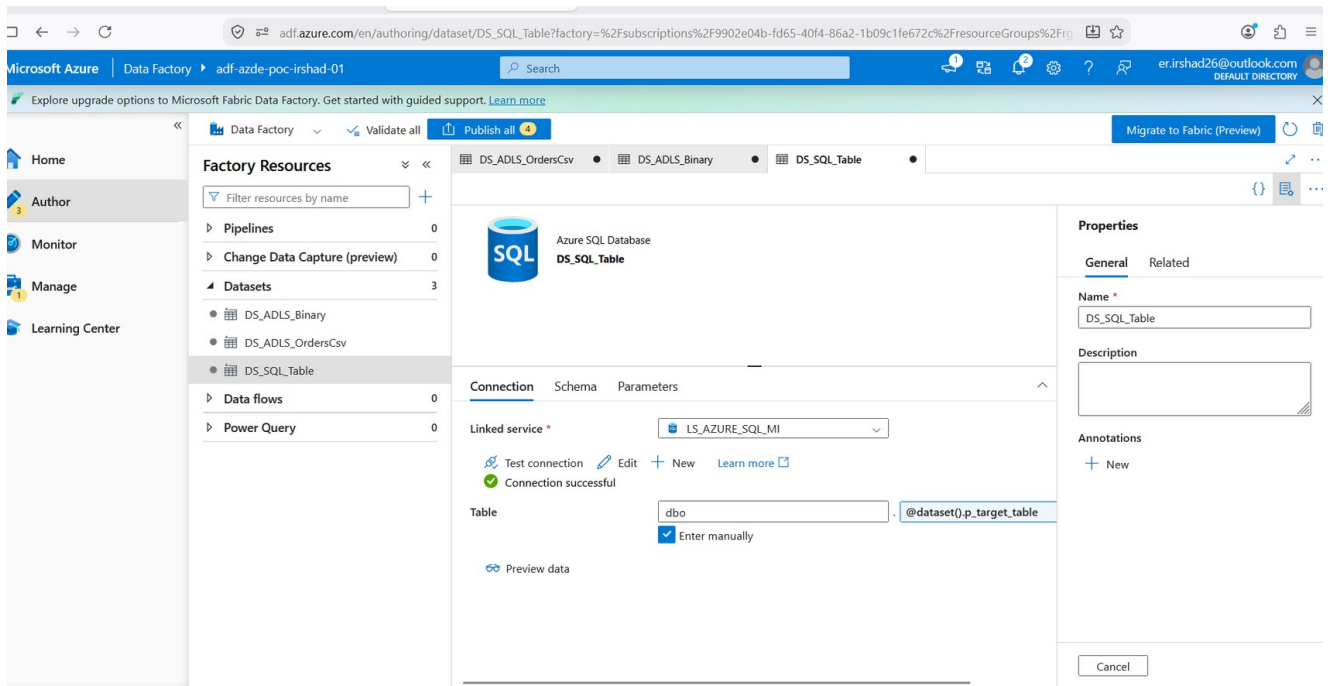


Figure A-22. Parameterized DS_SQL_Table dataset configuration. [Source file: Screenshot 2026-08-28 174455.png]

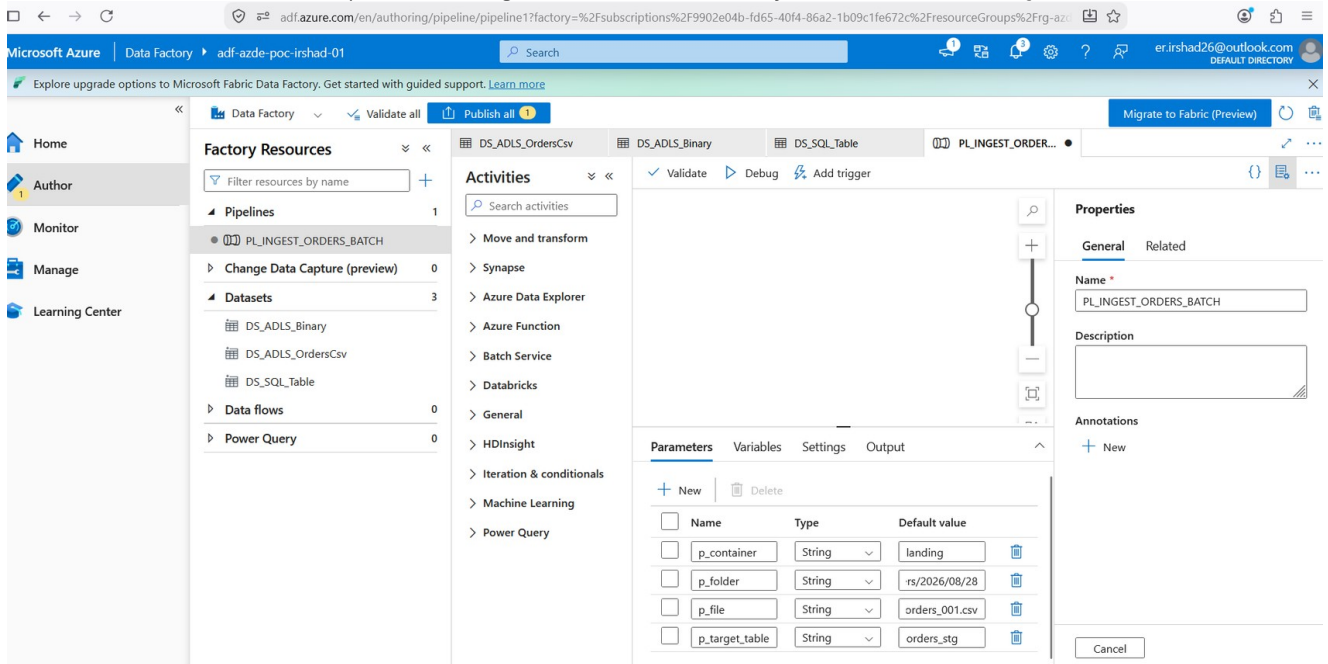


Figure A-23. `PL_INGEST_ORDERS_BATCH` pipeline parameters. [Source file: Screenshot 2026-08-28 174824.png]

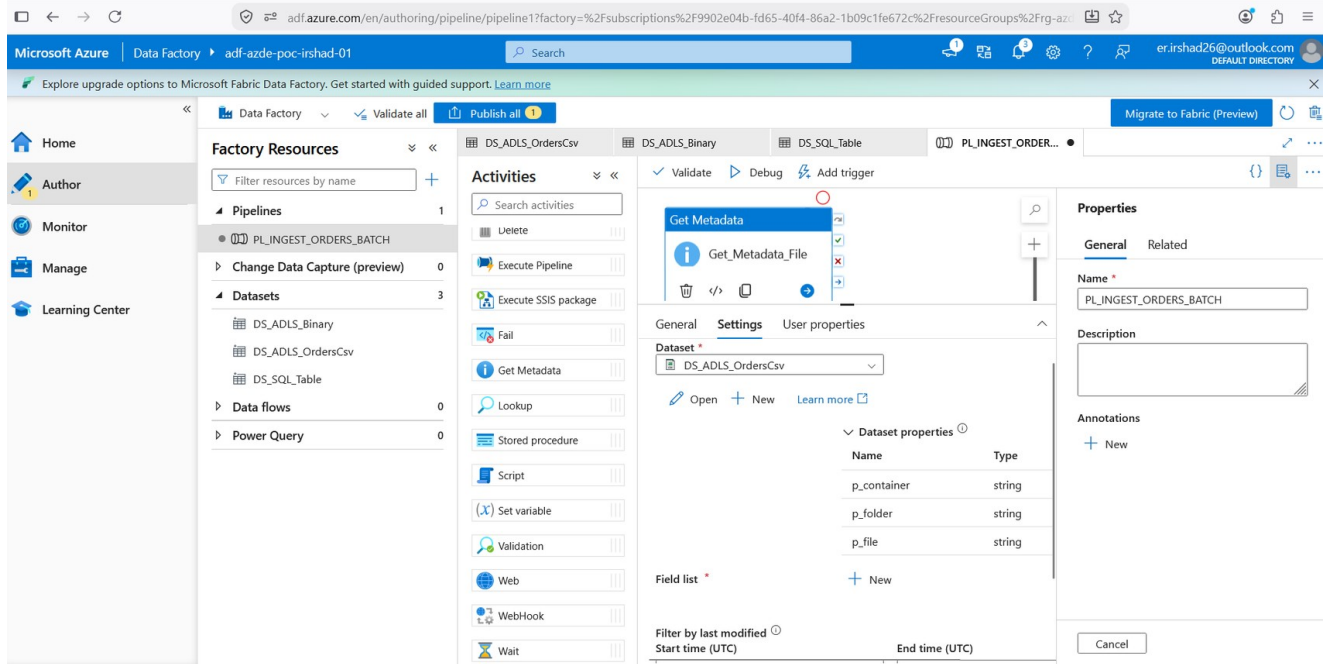


Figure A-24. `Get_Metadadata_File` activity configuration. [Source file: Screenshot 2026-08-28 180440.png]

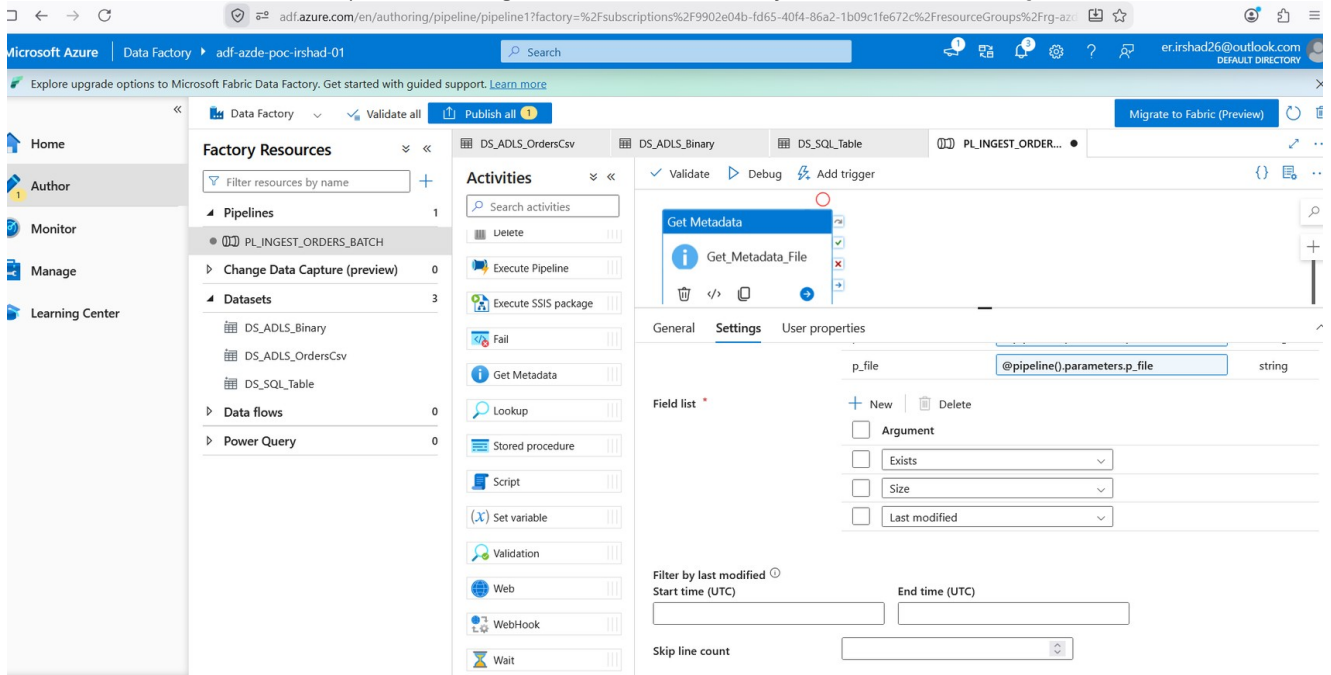


Figure A-25. Get Metadata activity field-list settings. [Source file: Screenshot 2026-08-28 181921.png]

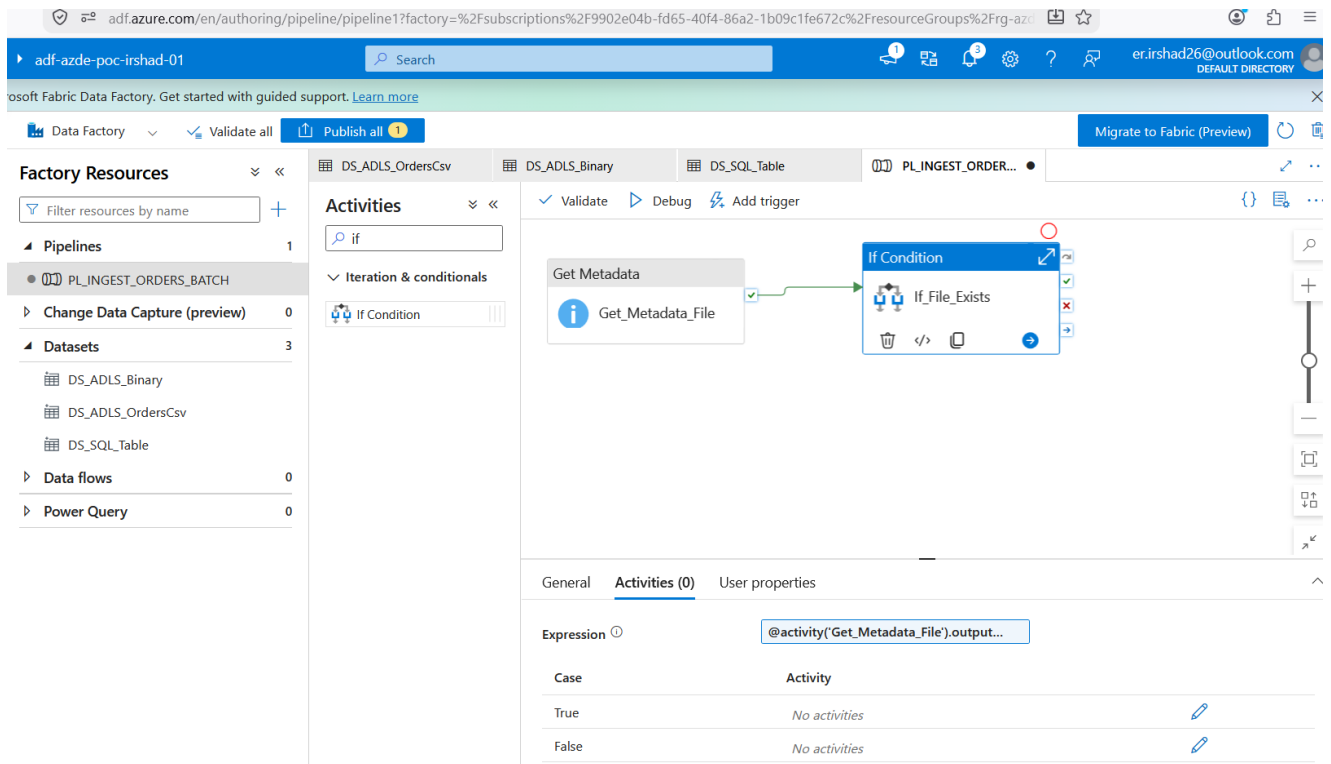


Figure A-26. If_File_Exists condition activity connected after Get Metadata. [Source file: Screenshot 2026-08-28 182608.png]

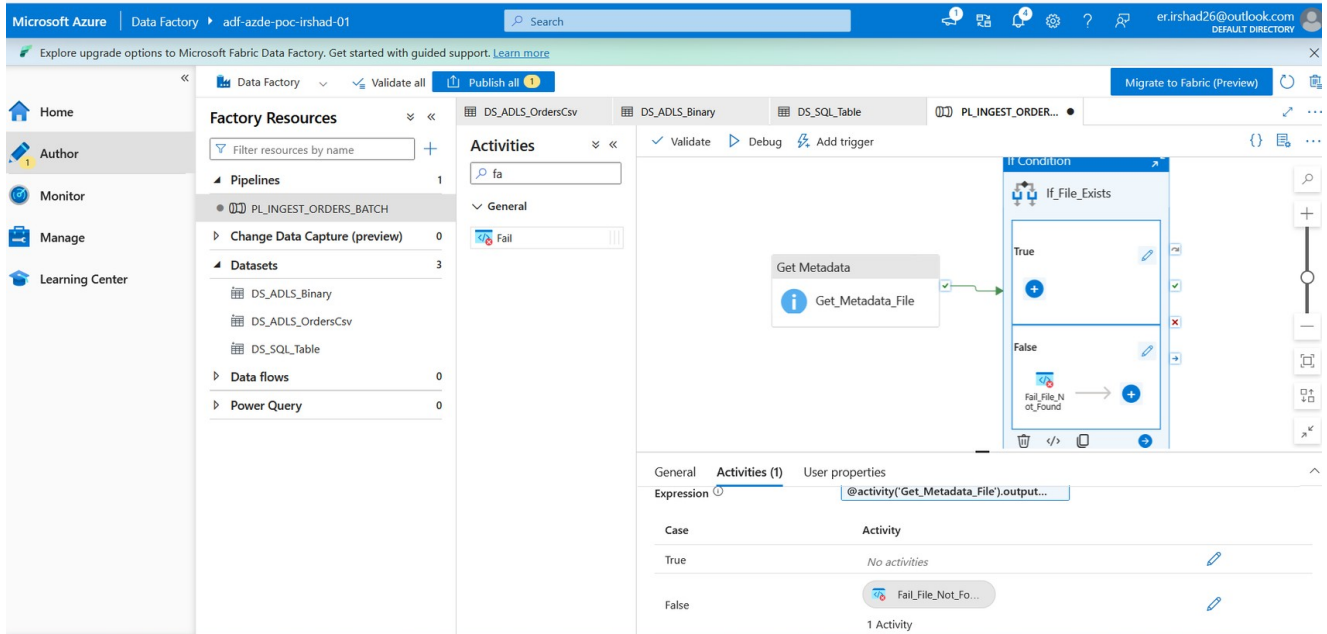


Figure A-27. If condition branch with Fail_File_Not_Found activity. [Source file: Screenshot 2026-08-28 184440.png]

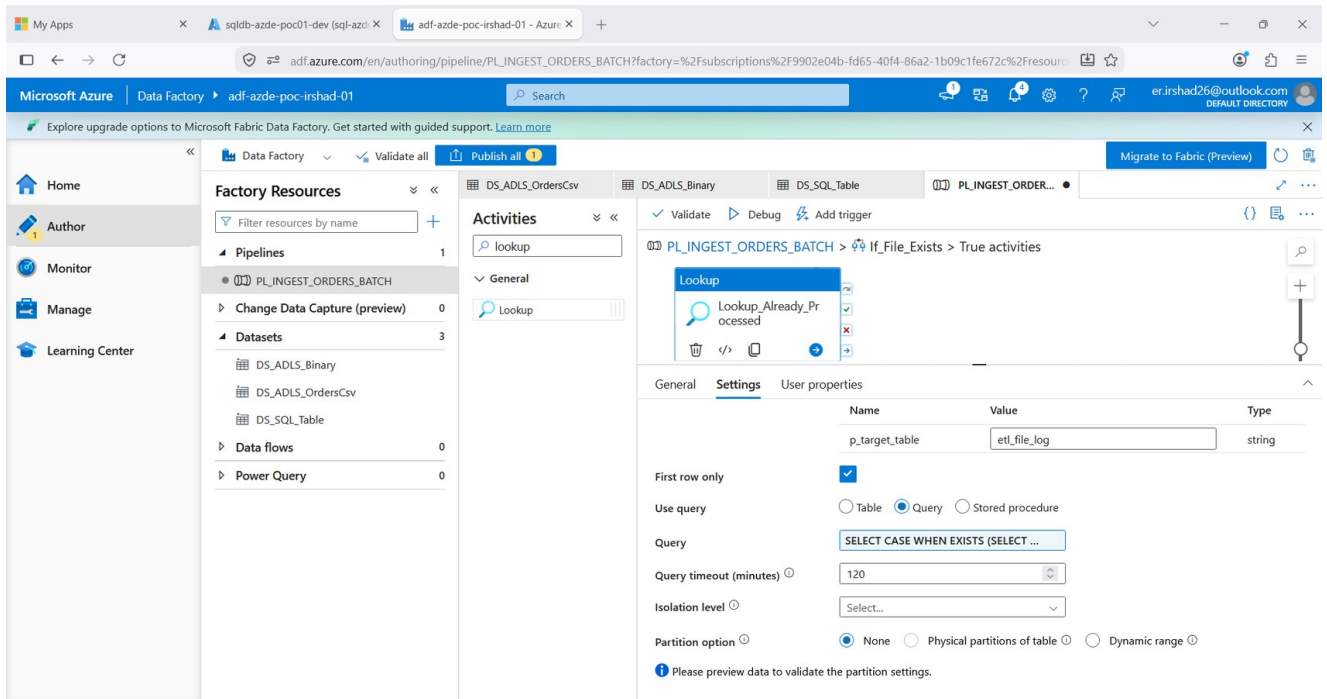


Figure A-28. ADF lookup/query configuration used during pipeline authoring. [Source file: Screenshot 2026-08-28 184930.png]

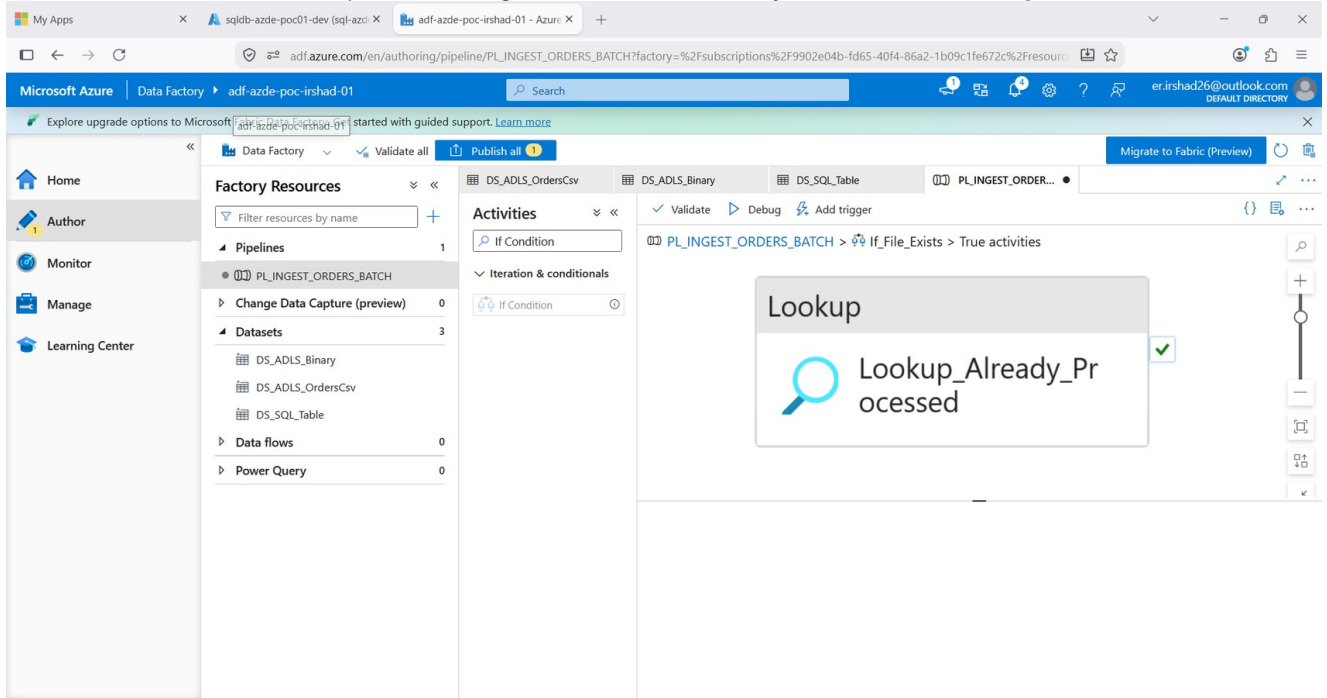


Figure A-29. Lookup activity visible on the ADF pipeline canvas. [Source file: Screenshot 2026-08-28 185530.png]

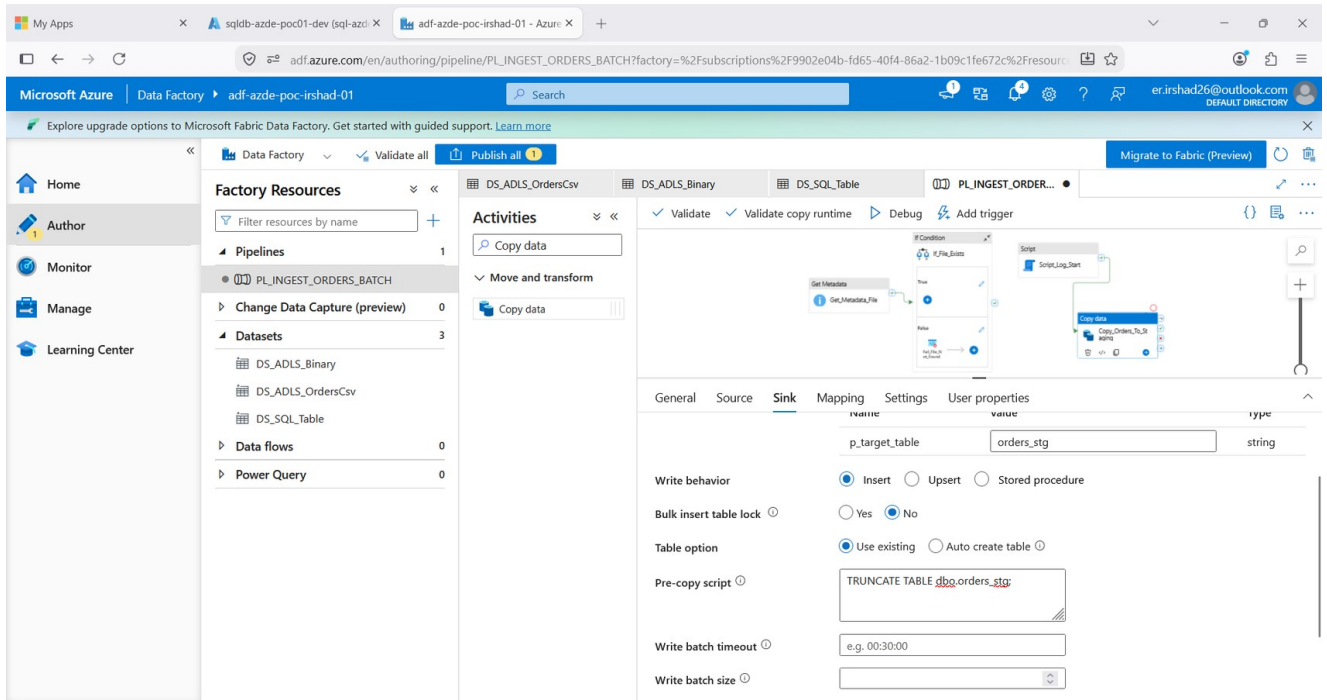


Figure A-30. Copy Activity settings including SQL staging/pre-copy configuration. [Source file: Screenshot 2026-08-28 190259.png]

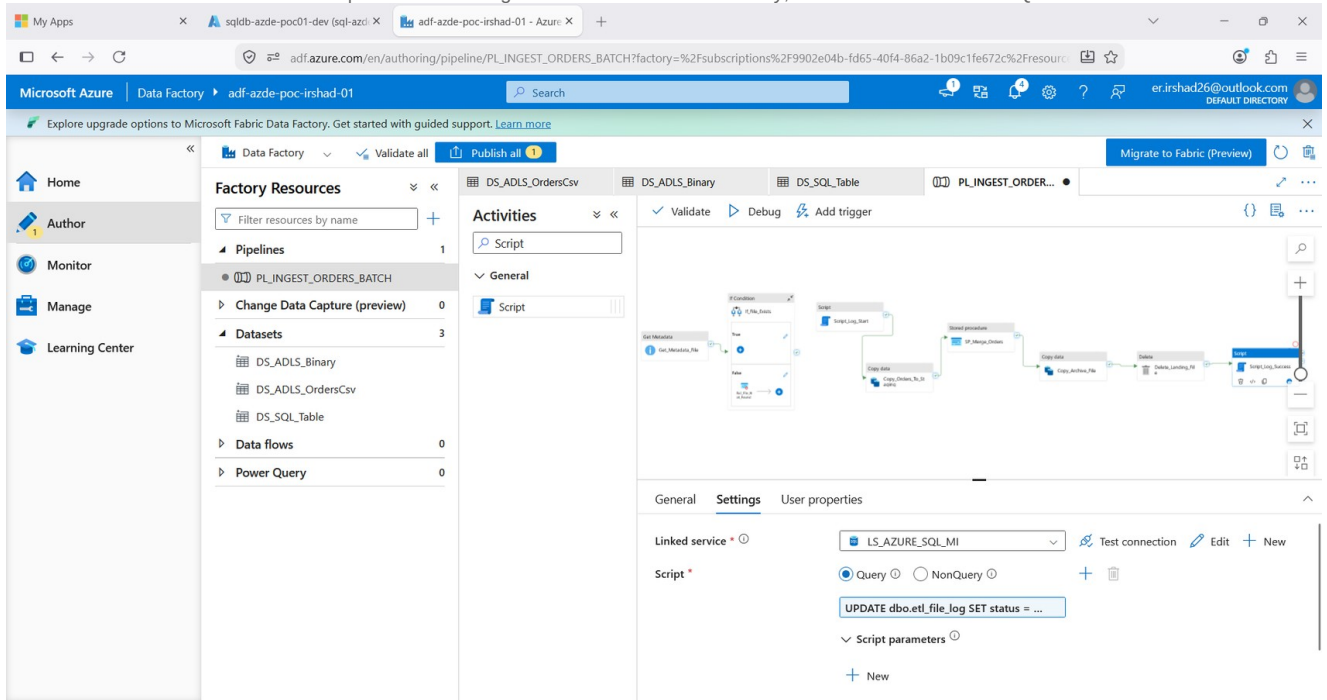


Figure A-31. ADF Script/SQL activity configuration in the pipeline. [Source file: Screenshot 2026-08-28 191511.png]

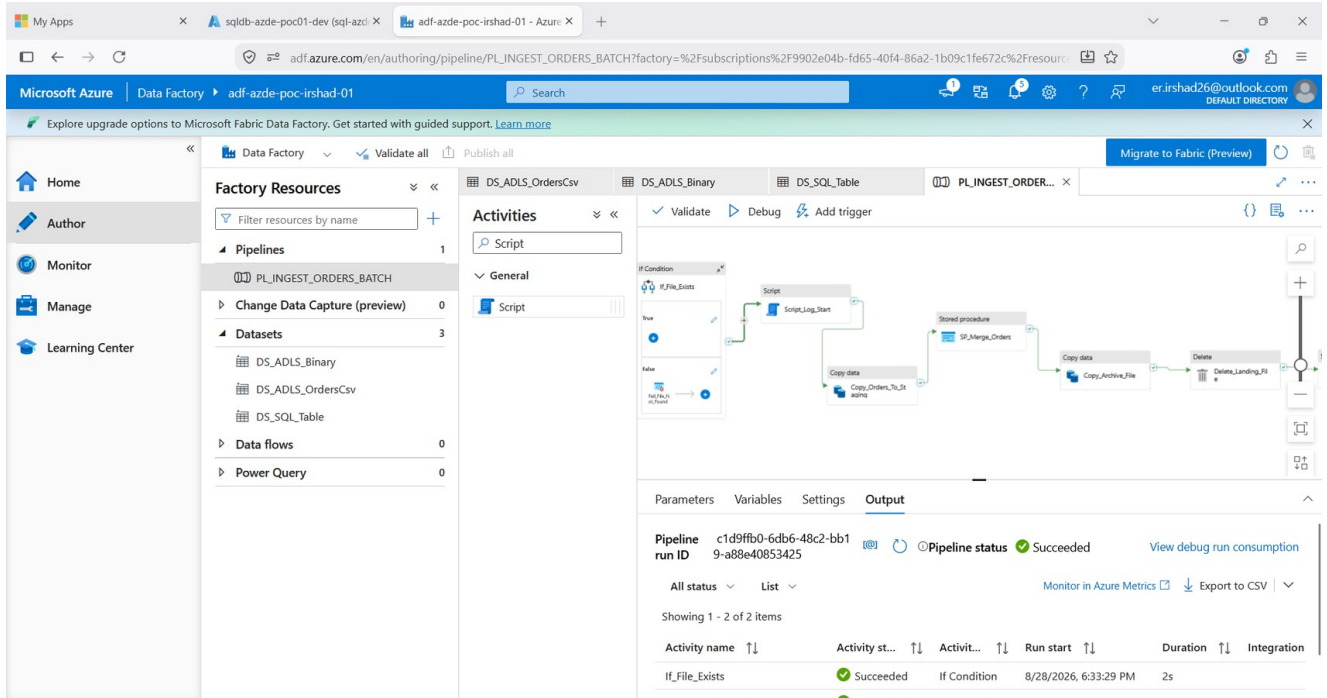


Figure A-32. ADF pipeline debug/output evidence during initial execution. [Source file: Screenshot 2026-08-28 192108.png]

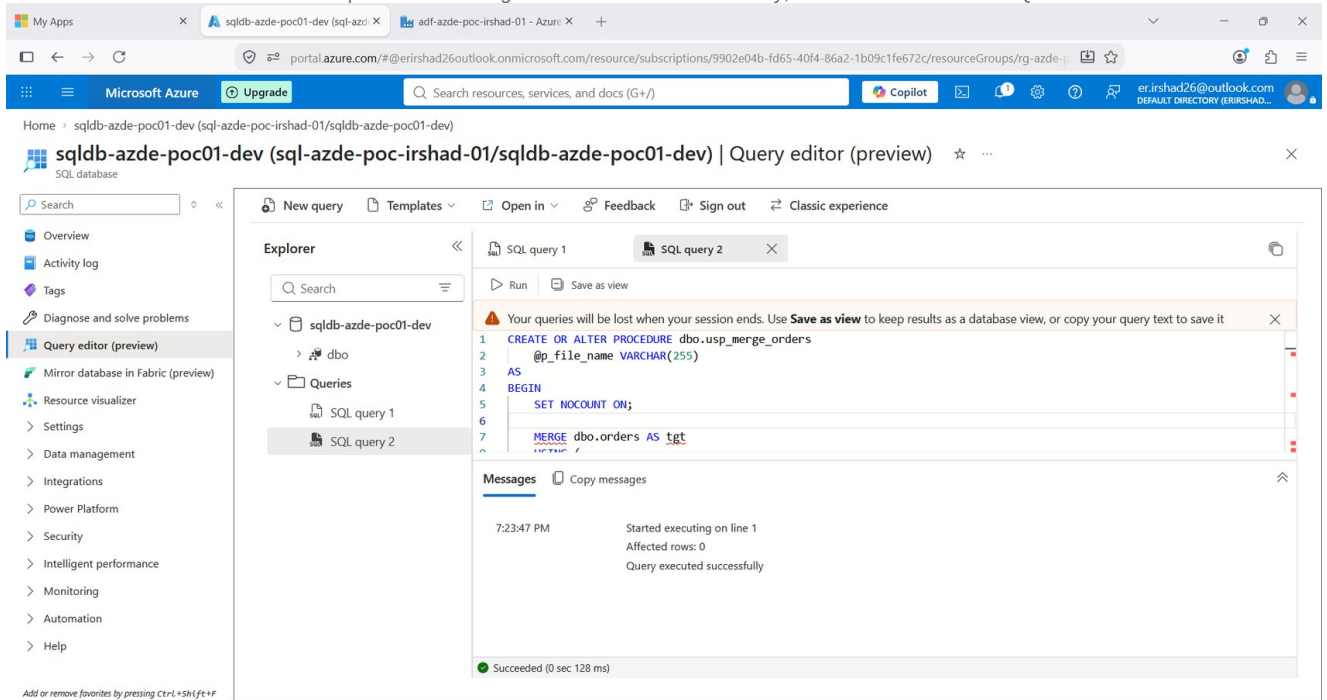


Figure A-33. Azure SQL Query Editor evidence during database object configuration/testing. [Source file: Screenshot 2026-08-28 192410.png]

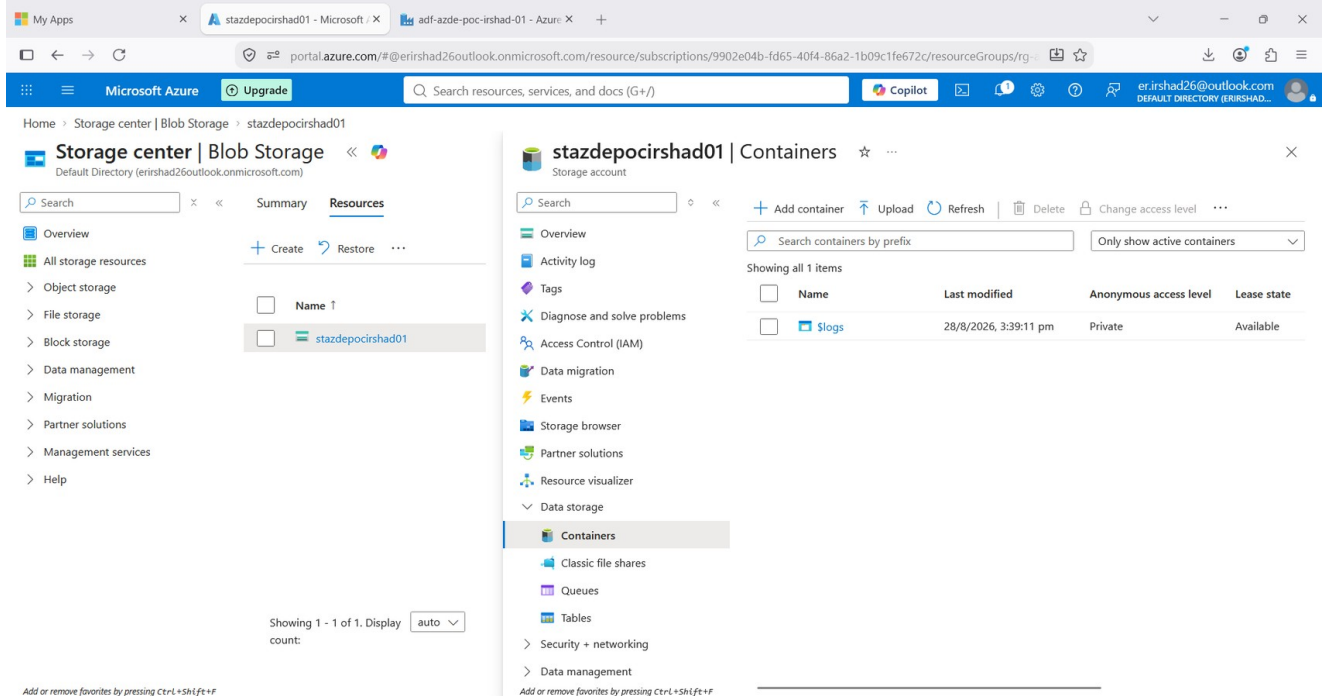


Figure A-34. ADLS storage/container verification during the pipeline test. [Source file: Screenshot 2026-08-28 192901.png]

The screenshot displays the Microsoft Azure Data Factory console. On the left, the 'Factory Resources' pane shows the pipeline 'PL_INGEST_ORDERS_BATCH' under the 'Pipelines' section. The main canvas shows the pipeline design with an 'If Condition' activity (containing 'If_File_Exists') and a 'Get Metadata' activity. The 'Output' pane at the bottom shows the pipeline run ID '2b5001d5-e489-4b82-b6a6-b4c5b90e0e24' with a status of 'In progress'. Below this, a table lists the activities:

Activity name	Activity st...	Activit...	Run start	Duration	Integration runtime
Get_Metadata_File	✓ Succeeded	Get Metadata	8/28/2026, 7:32:39 PM	30s	AutoResolveIntegrationRuntime (East US)

Figure A-35. ADF activity output showing an execution/authentication troubleshooting point. [Source file: Screenshot 2026-08-28 193332.png]

The screenshot displays the Microsoft Azure Data Factory console. On the left, the 'Factory Resources' pane shows the pipeline 'PL_INGEST_ORDERS_BATCH' under the 'Pipelines' section. The main canvas shows the pipeline design with an 'If Condition' activity (containing 'If_File_Exists') and a 'Get Metadata' activity. The 'Output' pane at the bottom shows the pipeline run ID '387fc975-bfdd-4b42-9209-d53ffcfd22' with a status of 'Failed'. Below this, a table lists the activities:

Activity name	Activity st...	Activit...	Run start	Duration	Integration
Fail_File_Not_Found	✗ Failed	Fail	8/30/2026, 10:26:32 AM	Less than 1s	
If_File_Exists	✗ Failed	If Condition	8/30/2026, 10:26:32 AM	2s	
Get_Metadata_File	✓ Succeeded	Get Metadata	8/30/2026, 10:26:13 AM	18s	AutoResolve

Figure A-36. ADF Monitor/debug evidence from a controlled failed run. [Source file: Screenshot 2026-08-30 102729.png]

The screenshot shows the Azure Data Factory Author view for a pipeline named 'PL_INGEST_ORDER...'. The pipeline is in a 'Failed' state. The 'Activities' pane on the right shows the execution details for the failed run.

Activity name	Activity status	Activity type	Run start	Duration	Integration
Script_Log_Start	Failed	Script	8/30/2026, 10:52:42 AM	1m 15s	AutoResolve
If_File_Exists	Succeeded	If Condition	8/30/2026, 10:52:40 AM	Less than 1s	
Get_Metadata_File	Succeeded	Get Metadata	8/30/2026, 10:52:31 AM	8s	AutoResolve

Figure A-37. ADF pipeline troubleshooting evidence with activity status details. [Source file: Screenshot 2026-08-30 105519.png]

The screenshot shows the same Azure Data Factory pipeline after a successful execution. The pipeline is now in a 'Succeeded' state. The 'Activities' pane on the right shows the execution details for the successful run.

Activity name	Activity status	Activity type	Run start	Duration	Integration
Script_Log_Success	Succeeded	Script	8/30/2026, 11:13:08 AM	12s	AutoResolve
Delete_Landing_File	Succeeded	Delete	8/30/2026, 11:12:45 AM	22s	AutoResolve
Copy_Archive_File	Succeeded	Copy data	8/30/2026, 11:12:27 AM	16s	AutoResolve
SP_Merge_Orders	Succeeded	Stored procedure	8/30/2026, 11:12:08 AM	18s	AutoResolve

Figure A-38. ADF successful activity output after fixing the pipeline configuration. [Source file: Screenshot 2026-08-30 111351.png]

SQL query 1

Run

Save as view

```
1  -- 1. Check curated/target table (should have 5 rows from orders_001.csv)
2  SELECT * FROM dbo.orders;
3
4  -- 2. Check staging table (should be truncated / 0 rows)
5  SELECT * FROM dbo.orders_stg;
6
7  -- 3. Check ETL file log table (status should be 'SUCCESS')
8  SELECT * FROM dbo.etl_file_log ORDER BY log_id DESC;
```

Messages

Results

	log_id	file_name	file_size...	status	rows_ins...	rows_up...	started_at	complete...	error_me...
1	1	orders_001.csv	276	SUCCESS	5	0	2026-08-30T0...	2026-08-30T0...	NULL

Figure A-39. Azure SQL verification queries and SUCCESS ETL log evidence. [Source file: Screenshot 2026-08-30 114112.png]

POC-01 | Secure Batch Ingestion with Azure Data Factory, ADLS Gen2 and Azure SQL

This PC > Windows (C:) > Program Files > Microsoft SDKs > Azure > CLI2 >

Name	Date modified	Type	Size
Lib	30-08-2026 15:00	File folder	
Scripts	30-08-2026 15:01	File folder	
wbin	30-08-2026 15:01	File folder	
_asyncio	10-06-2026 11:30	Python Extension ...	77 KB
_bz2	10-06-2026 11:30	Python Extension ...	86 KB
_ctypes	10-06-2026 11:30	Python Extension ...	140 KB
_decimal	10-06-2026 11:30	Python Extension ...	284 KB
_elementtree	10-06-2026 11:30	Python Extension ...	136 KB
_hashlib	10-06-2026 11:30	Python Extension ...	68 KB
_lzma	10-06-2026 11:30	Python Extension ...	158 KB
_multiprocessing	10-06-2026 11:30	Python Extension ...	38 KB
_overlapped	10-06-2026 11:30	Python Extension ...	57 KB
_queue	10-06-2026 11:30	Python Extension ...	36 KB
_remote_debugging	10-06-2026 11:30	Python Extension ...	83 KB
_socket	10-06-2026 11:30	Python Extension ...	86 KB
_sqlite3	10-06-2026 11:30	Python Extension ...	130 KB
_ssl	10-06-2026 11:30	Python Extension ...	187 KB
_uuid	10-06-2026 11:30	Python Extension ...	28 KB
_wmi	10-06-2026 11:30	Python Extension ...	40 KB
_zoneinfo	10-06-2026 11:30	Python Extension ...	51 KB
_zstd	10-06-2026 11:30	Python Extension ...	492 KB

Figure A-40. Windows Azure CLI installation directory evidence. [Source file: Screenshot 2026-08-30 150837.png]

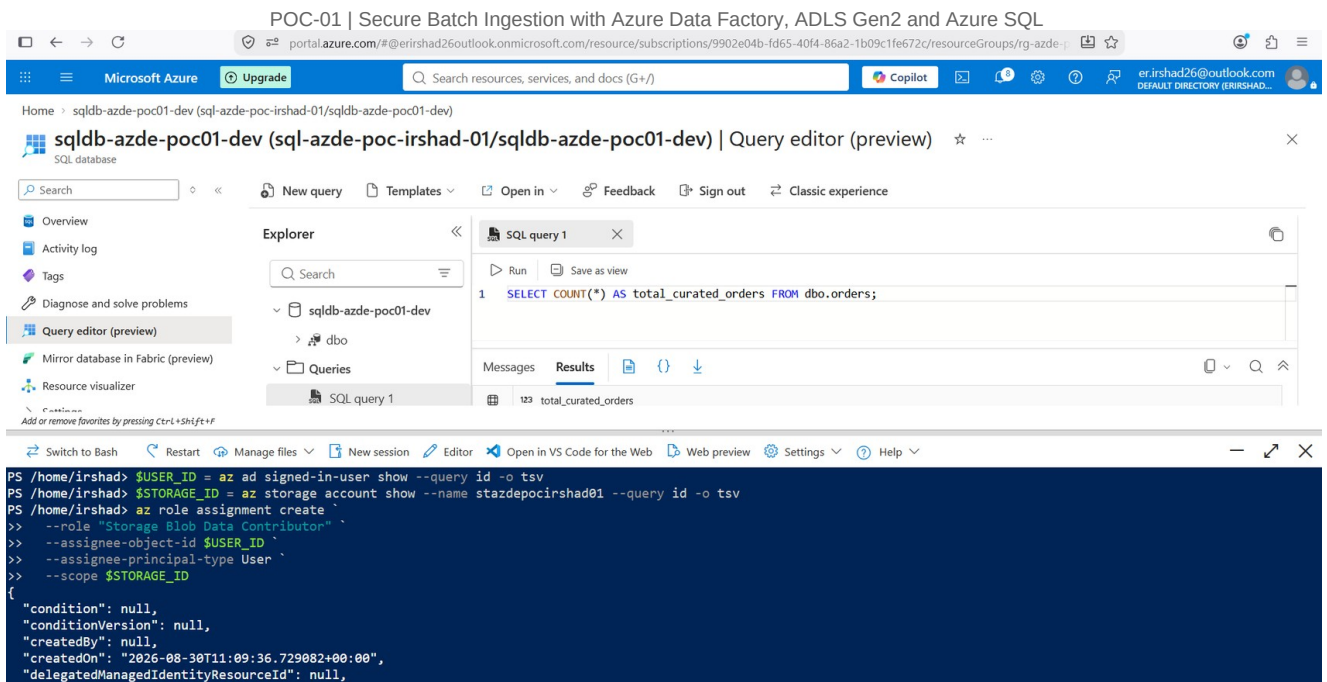


Figure A-41. Azure CLI login/account output together with SQL verification evidence. [Source file: Screenshot 2026-08-30 164425.png]

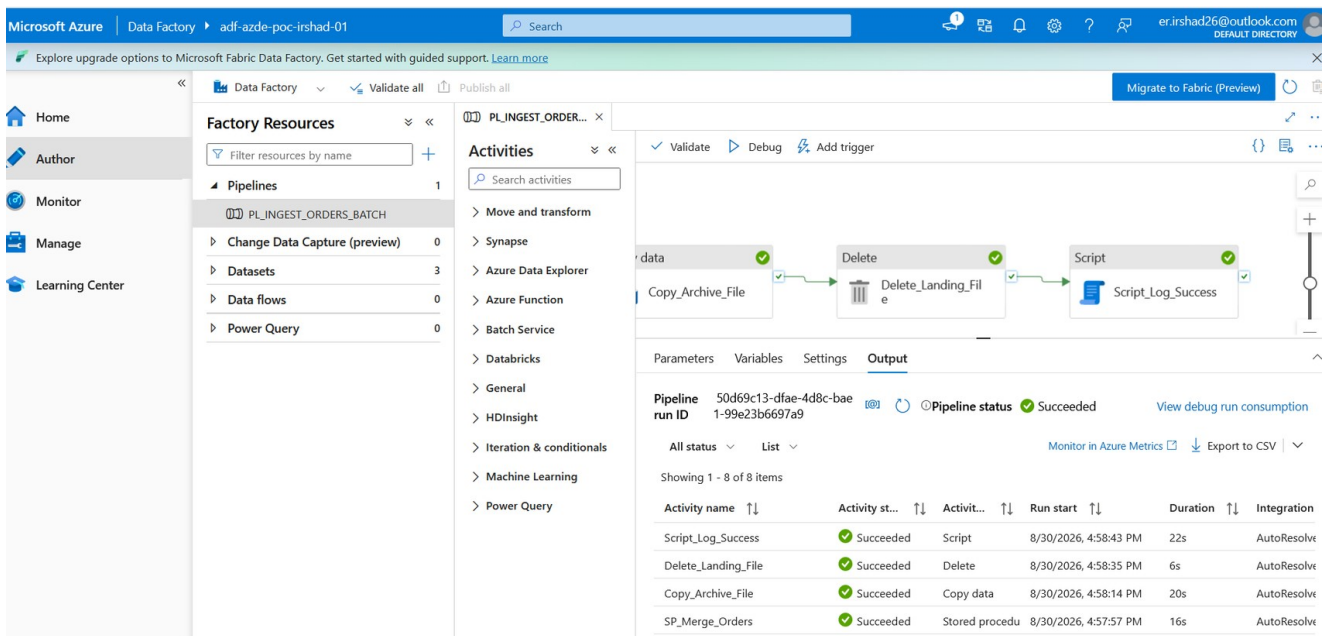


Figure A-42. Final successful ADF pipeline run with completed activities. [Source file: Screenshot 2026-08-30 171911.png]

Appendix B. Terraform source files from the POC package

The following source is reproduced from the supplied POC Terraform folder so the PDF remains useful even when read without the ZIP. Real local secrets must remain outside source control.

versions.tf

```
terraform {
  required_version = ">= 1.6.0"

  required_providers {
    azurerm = {
      source = "hashicorp/azurerm"
      version = "~> 4.0"
    }
  }
}

provider "azurerm" {
  features {}
  subscription_id = var.subscription_id
}
```

variables.tf

```
variable "subscription_id" {
  description = "Azure subscription ID. Prefer ARM_SUBSCRIPTION_ID environment variable when possible."
  type        = string
  default     = null
}

variable "location" {
  description = "Azure region for POC resources."
  type        = string
  default     = "centralindia"
}

variable "resource_group_name" {
  type        = string
  default     = "rg-azde-poc01-dev"
}

variable "storage_account_name" {
  description = "Globally unique, lowercase letters/numbers only."
  type        = string
}

variable "data_factory_name" {
  description = "Globally unique Data Factory name."
  type        = string
}

variable "sql_server_name" {
  description = "Globally unique Azure SQL logical server name (without .database.windows.net)."
  type        = string
}

variable "sql_database_name" {
  type        = string
  default     = "sqladb-azde-poc01-dev"
}

variable "sql_admin_login" {
  description = "Bootstrap SQL admin login. Do not commit the real value."
  type        = string
  sensitive   = true
}

variable "sql_admin_password" {
  description = "Bootstrap SQL admin password. Do not commit the real value."
  type        = string
  sensitive   = true
}

variable "key_vault_name" {
  description = "Globally unique Key Vault name."
  type        = string
}

variable "client_ip" {
  description = "Optional public IPv4 address of your laptop for SQL admin bootstrap access."
  type        = string
  default     = null
}

variable "uploader_object_id" {
  description = "Optional Microsoft Entra object ID of your developer user. If set, grants Storage Blob Data Contributor for Python upload."
  type        = string
  default     = null
}

variable "create_log_analytics" {
  description = "Create a small Log Analytics workspace for optional diagnostics practice."
  type        = bool
  default     = false
}
```

```
variable "tags" {
  type = map(string)
  default = {
    project      = "azure-poc"
    environment  = "dev"
    owner        = "personal"
    autoDelete   = "true"
  }
}
```

main.tf

```
data "azurerm_client_config" "current" {}

resource "azurerm_resource_group" "this" {
  name     = var.resource_group_name
  location = var.location
  tags     = var.tags
}

resource "azurerm_storage_account" "this" {
  name                        = var.storage_account_name
  resource_group_name        = azurerm_resource_group.this.name
  location                   = azurerm_resource_group.this.location
  account_tier                = "Standard"
  account_replication_type    = "LRS"
  account_kind                = "StorageV2"
  is_hns_enabled              = true
  min_tls_version             = "TLS1_2"

  allow_nested_items_to_be_public = false
  public_network_access_enabled   = true

  tags = var.tags
}

resource "azurerm_storage_container" "landing" {
  name                = "landing"
  storage_account_id = azurerm_storage_account.this.id
  container_access_type = "private"
}

resource "azurerm_storage_container" "archive" {
  name                = "archive"
  storage_account_id = azurerm_storage_account.this.id
  container_access_type = "private"
}

resource "azurerm_storage_container" "quarantine" {
  name                = "quarantine"
  storage_account_id = azurerm_storage_account.this.id
  container_access_type = "private"
}

resource "azurerm_data_factory" "this" {
  name             = var.data_factory_name
  location          = azurerm_resource_group.this.location
  resource_group_name = azurerm_resource_group.this.name

  identity {
    type = "SystemAssigned"
  }

  tags = var.tags
}

resource "azurerm_role_assignment" "adf_storage" {
  scope                = azurerm_storage_account.this.id
  role_definition_name = "Storage Blob Data Contributor"
  principal_id         = azurerm_data_factory.this.identity[0].principal_id
}

resource "azurerm_role_assignment" "developer_storage" {
  count = var.uploader_object_id == null ? 0 : 1
  scope                = azurerm_storage_account.this.id
  role_definition_name = "Storage Blob Data Contributor"
  principal_id         = var.uploader_object_id
}

resource "azurerm_key_vault" "this" {
  name                = var.key_vault_name
  location             = azurerm_resource_group.this.location
  resource_group_name = azurerm_resource_group.this.name
  tenant_id           = data.azurerm_client_config.current.tenant_id
  sku_name            = "standard"
  enable_rbac_authorization = true
  soft_delete_retention_days = 7

  public_network_access_enabled = true

  tags = var.tags
}

# The main POC data path does not need a Key Vault secret because Managed Identity is preferred.
# This role lets the ADF identity read secrets later if you intentionally add a connector that requires one.
resource "azurerm_role_assignment" "adf_key_vault" {
  scope                = azurerm_key_vault.this.id
  role_definition_name = "Key Vault Secrets User"
  principal_id         = azurerm_data_factory.this.identity[0].principal_id
}

resource "azurerm_mssql_server" "this" {
  name = var.sql_server_name
}
```

```

resource_group_name = azurerm_resource_group.this.name
location            = azurerm_resource_group.this.location
version            = "12.0"
administrator_login = var.sql_admin_login
administrator_login_password = var.sql_admin_password
minimum_tls_version = "1.2"
public_network_access_enabled = true

tags = var.tags
}

resource "azurerm_mssql_database" "this" {
  name = var.sql_database_name
  server_id = azurerm_mssql_server.this.id
  sku_name = "Basic"
  max_size_gb = 2

  tags = var.tags
}

# Beginner-lab networking compromise: lets Azure services such as ADF Azure IR reach Azure SQL.
# Replace this with managed private endpoints / Private Link in the hardened version.
resource "azurerm_mssql_firewall_rule" "allow_azure_services" {
  name = "AllowAzureServices"
  server_id = azurerm_mssql_server.this.id
  start_ip_address = "0.0.0.0"
  end_ip_address = "0.0.0.0"
}

resource "azurerm_mssql_firewall_rule" "developer_ip" {
  count = var.client_ip == null ? 0 : 1
  name = "DeveloperLaptop"
  server_id = azurerm_mssql_server.this.id
  start_ip_address = var.client_ip
  end_ip_address = var.client_ip
}

resource "azurerm_log_analytics_workspace" "this" {
  count = var.create_log_analytics ? 1 : 0
  name = "law-azde-poc01-dev"
  location = azurerm_resource_group.this.location
  resource_group_name = azurerm_resource_group.this.name
  sku = "PerGB2018"
  retention_in_days = 30

  tags = var.tags
}

```

outputs.tf

```

output "resource_group_name" {
  value = azurerm_resource_group.this.name
}

output "storage_account_name" {
  value = azurerm_storage_account.this.name
}

output "storage_dfs_url" {
  value = azurerm_storage_account.this.primary_dfs_endpoint
}

output "data_factory_name" {
  value = azurerm_data_factory.this.name
}

output "data_factory_managed_identity_object_id" {
  value = azurerm_data_factory.this.identity[0].principal_id
}

output "sql_server_fqdn" {
  value = azurerm_mssql_server.this.fully_qualified_domain_name
}

output "sql_database_name" {
  value = azurerm_mssql_database.this.name
}

output "key_vault_uri" {
  value = azurerm_key_vault.this.vault_uri
}

```

terraform.tfvars.example

```

subscription_id = "<YOUR_SUBSCRIPTION_ID>"
location        = "centralindia"
resource_group_name = "rg-azde-poc01-dev"

storage_account_name = "stazdepoc01CHANGE"
data_factory_name    = "adf-azde-poc01-CHANGE"
sql_server_name      = "sql-azde-poc01-CHANGE"
sql_database_name    = "sqldb-azde-poc01-dev"
key_vault_name       = "kv-azde-poc01-CHANGE"

```

```
# NEVER commit real credentials. Copy this file to terraform.tfvars locally.
sql_admin_login      = "pocsqladmin"
sql_admin_password   = "CHANGE_ME_LOCALLY"

# Optional: your current public IPv4 address for SQL client access.
client_ip = null

# Optional: your Microsoft Entra object ID so the local Python uploader has blob data access.
uploader_object_id = null

create_log_analytics = false
```

Final POC completion checklist

- [x] Resource Group and cost guardrail created
- [x] ADLS Gen2 account and landing/archive/quarantine created
- [x] Azure SQL logical server/database created
- [x] SQL firewall configured for beginner lab
- [x] Microsoft Entra SQL admin configured
- [x] ADF created with system-assigned Managed Identity
- [x] ADF Storage Blob Data Contributor assigned
- [x] ADF SQL contained user/permissions created
- [x] Linked services and parameterized datasets created
- [x] PL_INGEST_ORDERS_BATCH built, validated and published
- [x] Python/Azure CLI upload path tested
- [x] First run succeeds and curated count is 5
- [x] Archive/delete behavior verified
- [x] Same-file rerun remains 5 curated rows
- [x] Missing-file controlled failure verified
- [x] RBAC failure and recovery verified
- [x] ADF Monitor and SQL evidence captured
- [x] Terraform plan/apply workflow documented
- [x] Cleanup approach documented

Pass condition: The manual POC is complete. Keep this PDF and the sanitized project repository as the implementation record.